

CI012 - Lo Sviluppo su Business Cube (su Bus .NET Framework)

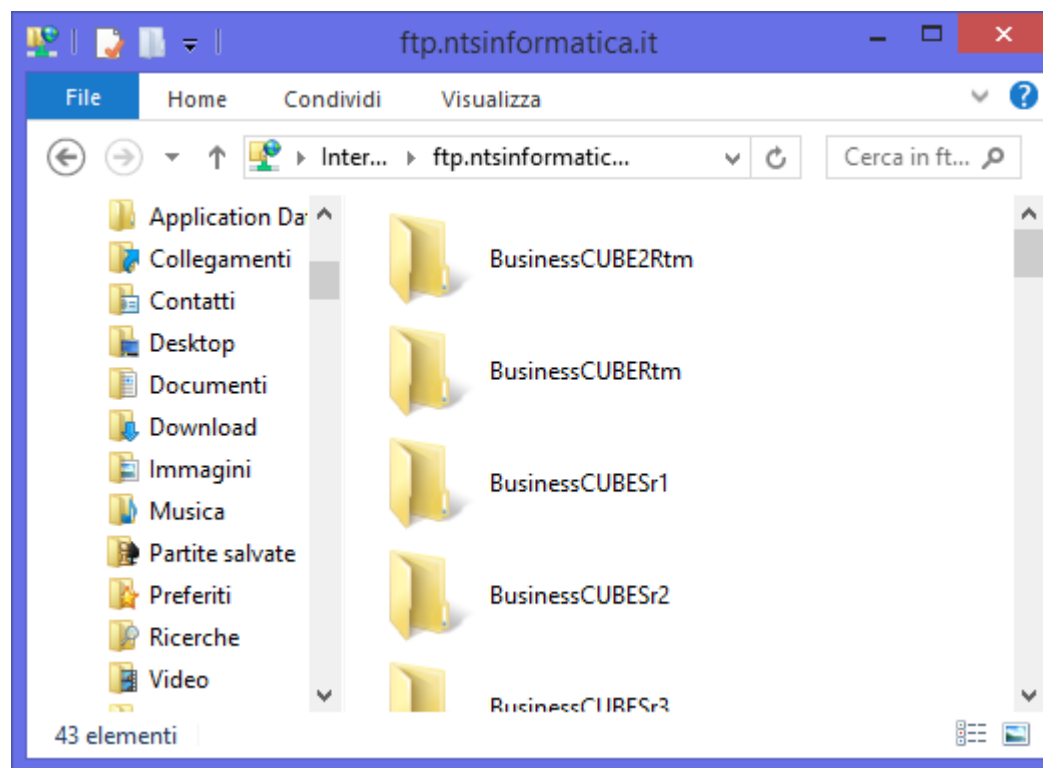
FABRIZIO FILIPPUCCI

Cosa ci serve per sviluppare ?

- **Business CUBE 2 sr 1 (cd + chiave di attivazione)**
- **Framework Microsoft .NET 4.5.2**
- **Runtime Crystal Report 2008 / 2011 (installato durante l'installazione di Business CUBE) NB: se vogliamo modificare o creare report ci occorrerà avere anche il relativo editor**
- **Microsoft Visual Studio 2015 / 2017 / 2019**
- **DevExpress suite DXperience professional V.14.2.3.0**

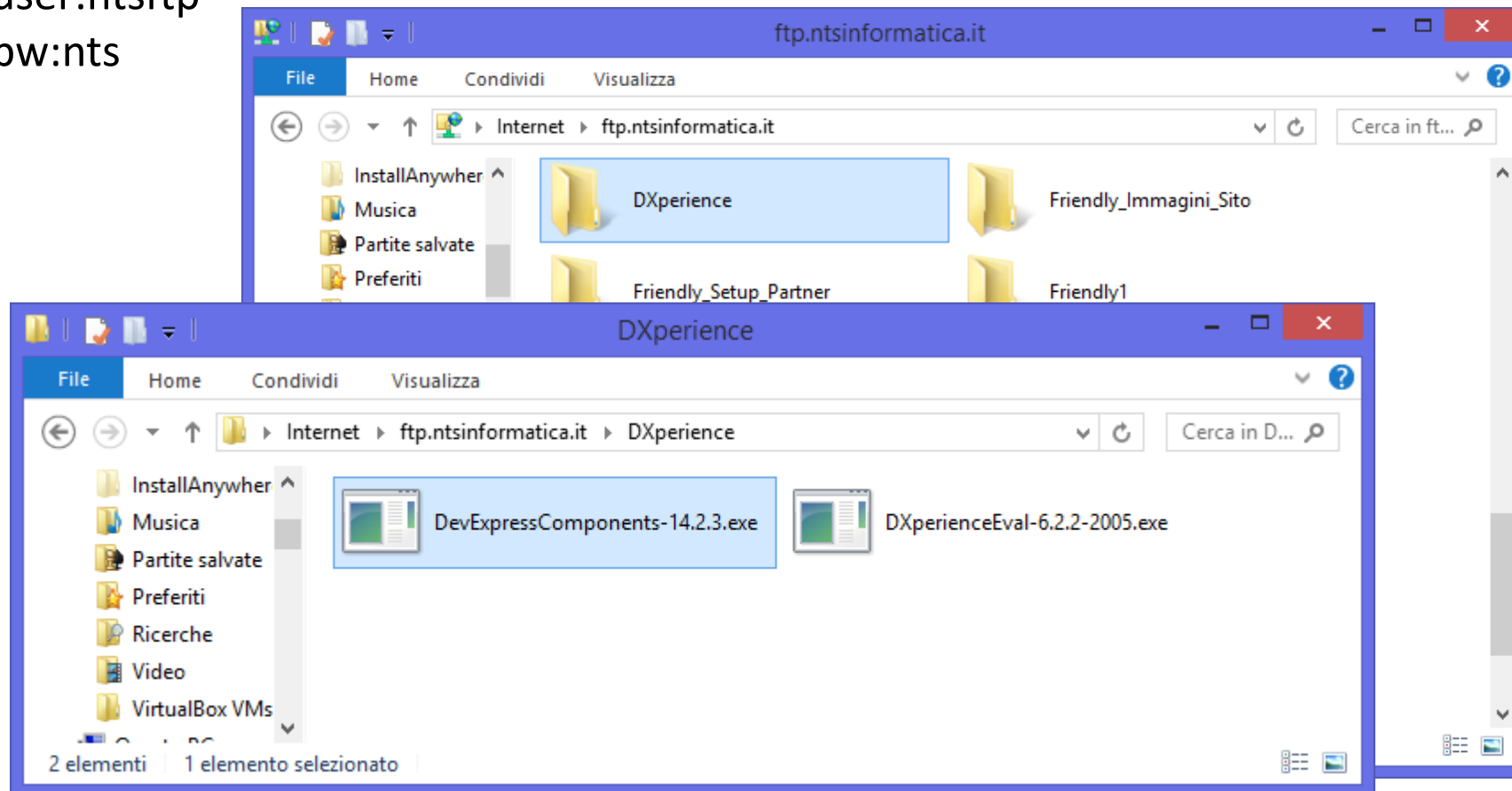


- ftp://ftp.ntsinformatica.it/
- user:ntsftp
- pw:nts



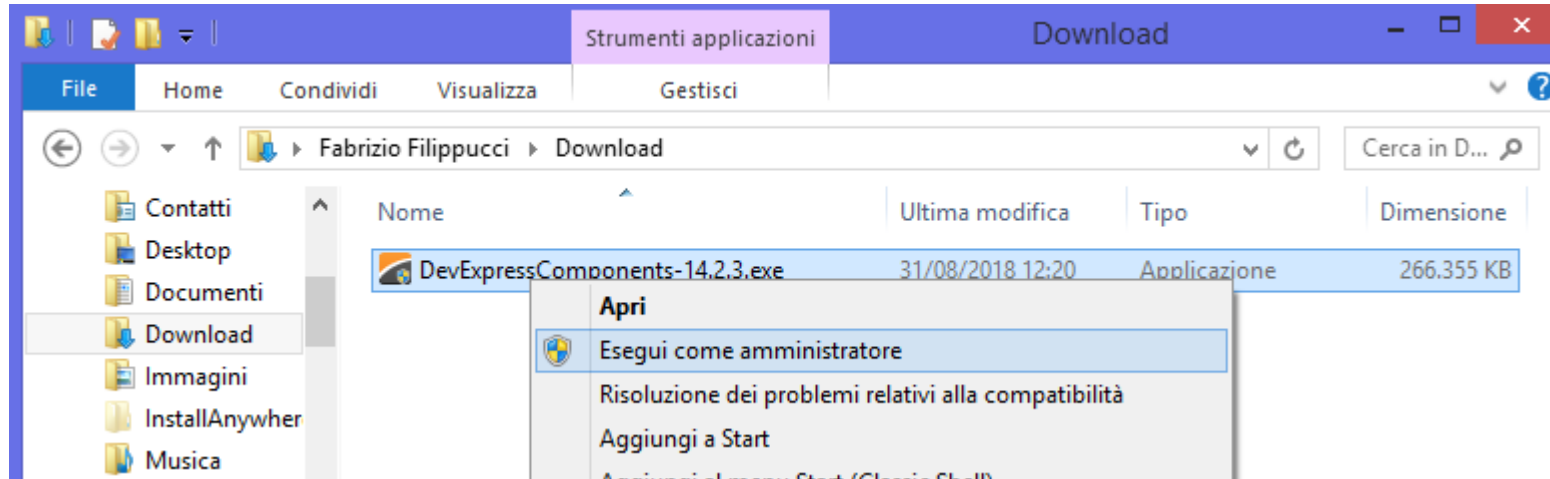
Installare la DevExpress

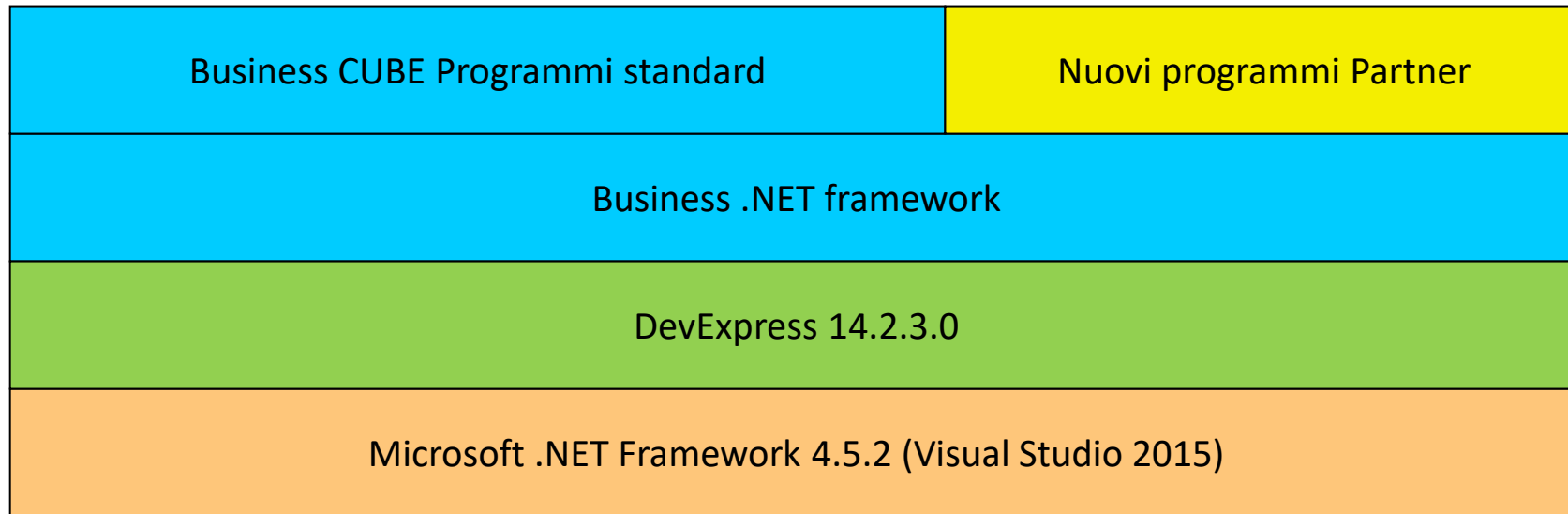
ftp://ftp.ntsinformatica.it/
user:ntsftp
pw:nts

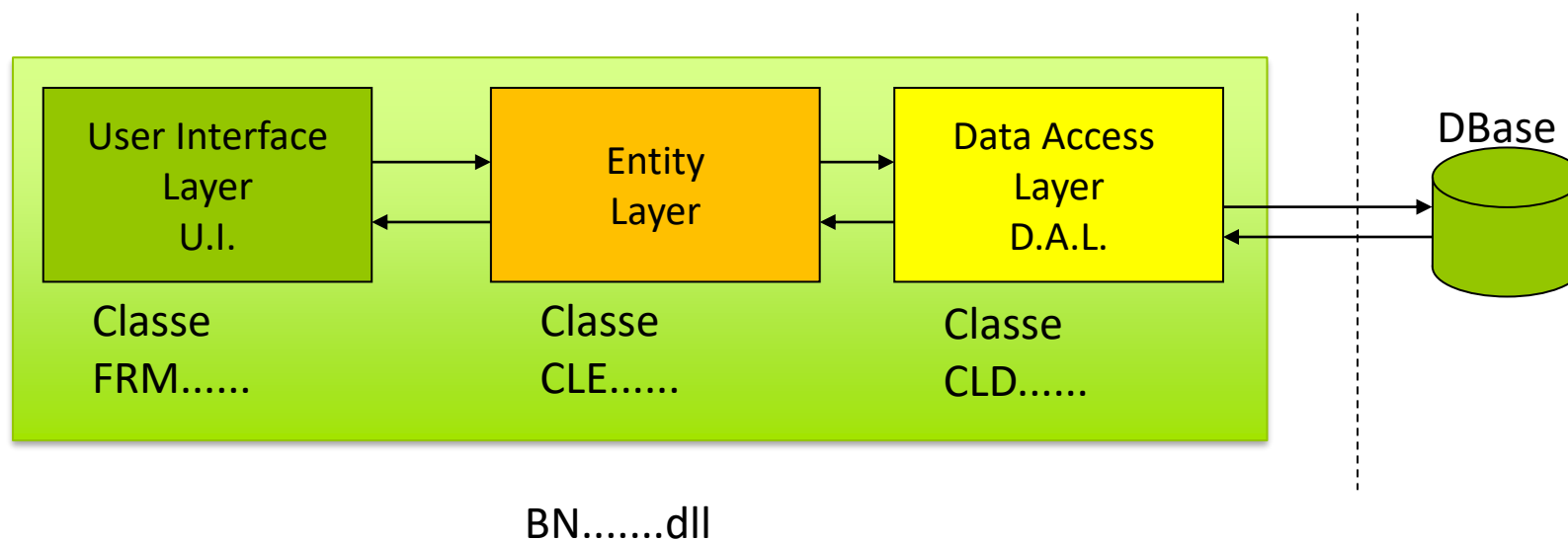


Installare la DevExpress

- Per installare i componenti DevExpress basta eseguire il file indicato







- Data Access Layer (D.A.L.):
 - Contiene query di database (sql)
 - Dovrebbe essere stateless

```
Public Class CLD__ABIA  
Inherits CLD__BASE
```


- Entity Layer:
 - Contiene controlli non formali dei dati
 - Contiene calcoli e logiche
 - Deve poter dialogare con diverse interfacce
 - E' 'statefull'

```
Public Class CLE__ABIA  
  Inherits CLE__BASE / CLE__BASN
```

- User Interface Layer (U.I.):
 - ‘Dialoga’ con l’utente
 - Contiene gli elementi grafici dell’applicazione
 - Può fare validazioni formali dei dati (es.: è un numero?)
 - E’ ‘statefull’

```
Public Class FRM__ABIA  
Inherits FRM__CHIL
```

Progetto 'CUBE' i REQUISITI INIZIALI

- Architettura multi-tier: separazione interfaccia – logica applicativa – accesso ai database
- Maggiore possibilità/facilità di realizzazione e di gestione di soluzioni verticali e custom
- Interfaccia utente più accattivante, più configurabile anche a livello grafico
- Utilizzo operativo più intuitivo
- Apertura ad interfaccia utente diversa da finestre Windows
- Superamento alcuni limiti esistenti nella 'localizzabilità' del software

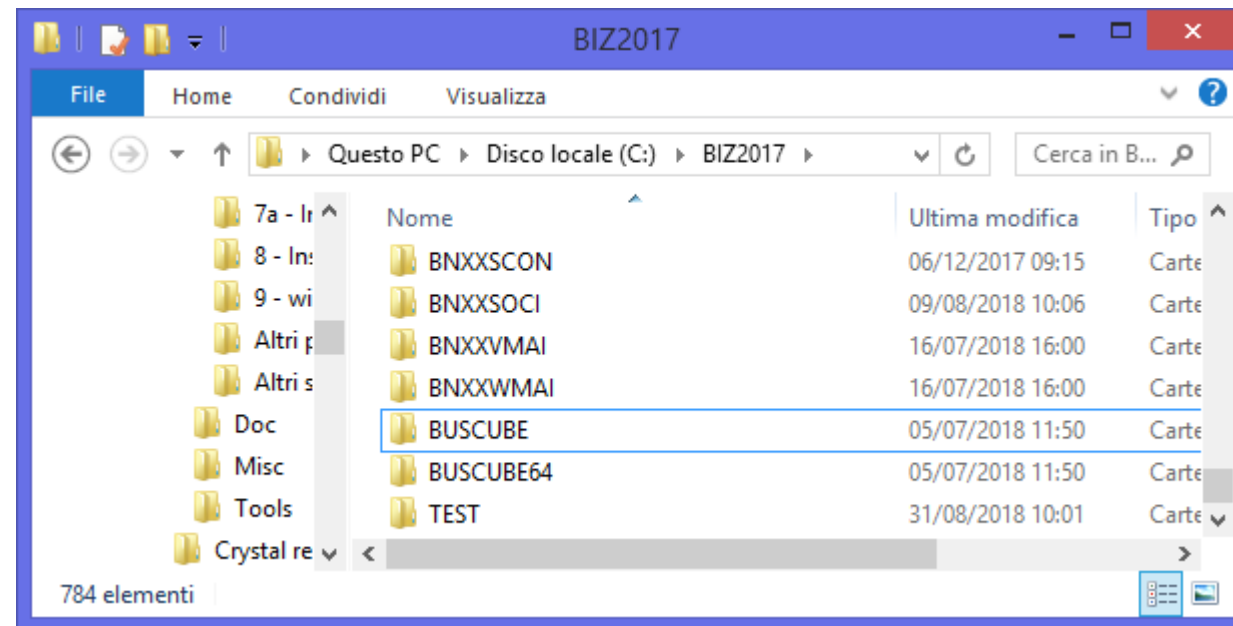


Costruire l'ambiente

- Per costruire l'ambiente occorre creare in un punto a scelta (non in c:\windows e non in c:\programmi) la cartella BIZ20xx
- Si consiglia di crearle nella route c:\ (o altro disco, non usare route troppo lunghe o contenenti spazi)

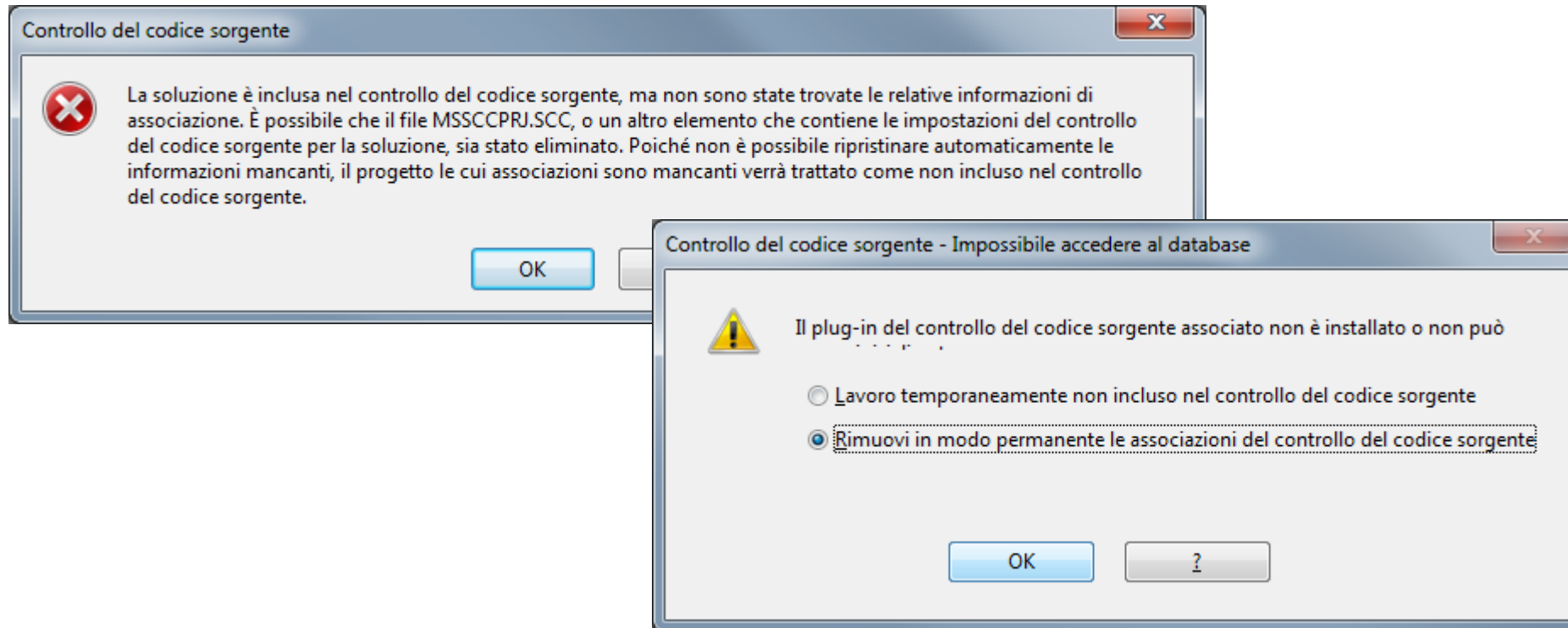
Per esempio:

Nel corso estrarre il pacchetto BIZ20xx.zip



Come trattare i sorgenti

- Caricare i sorgenti nella BIZ20xx e togliere la sola lettura (cartelle e file)
- All'apertura rimuovere i collegamenti a controllo del codice sorgente

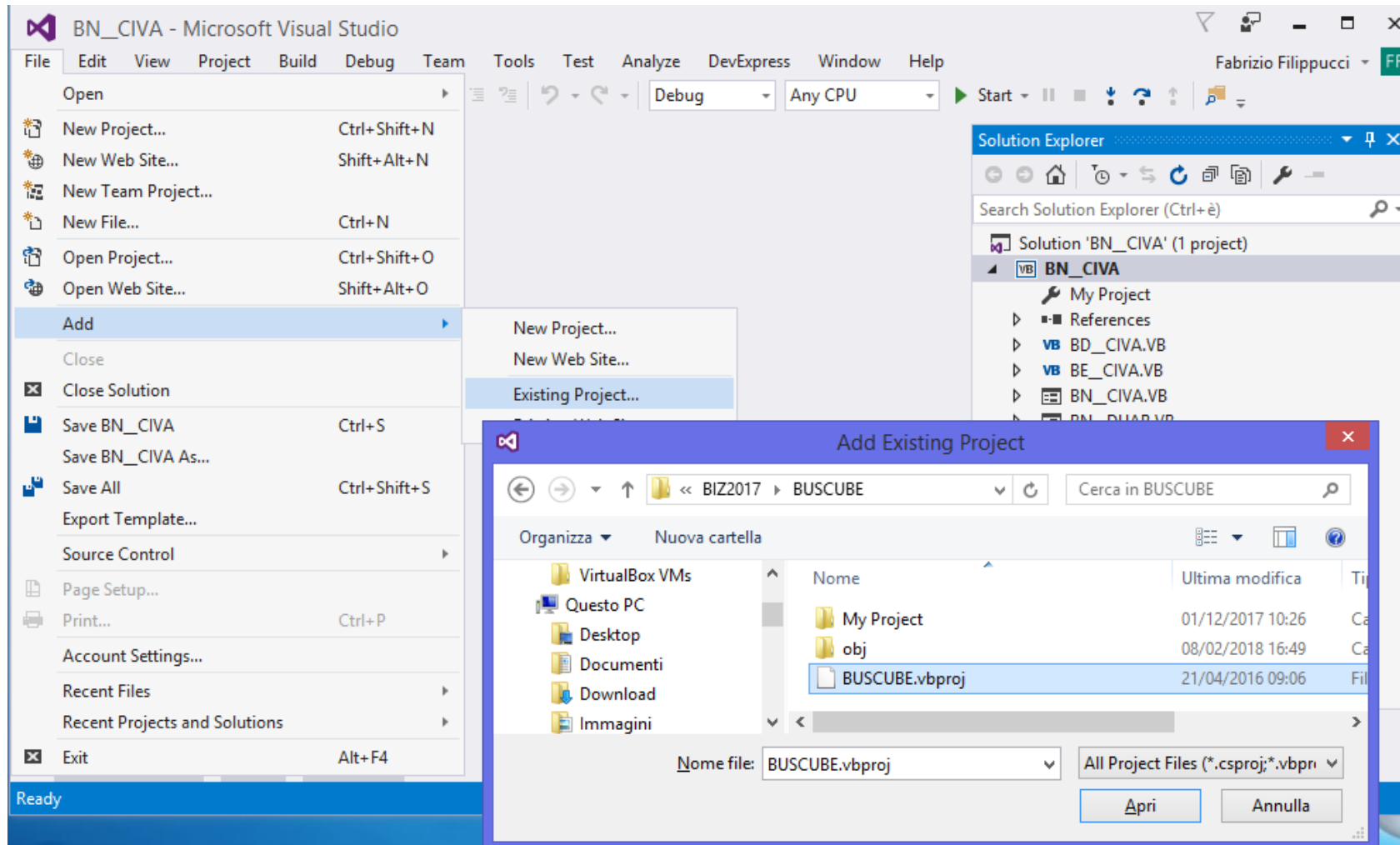


- All'apertura, se si lavora con una versione di visual studio successiva alla 2015, effettuare la conversione

The screenshot shows the Microsoft Visual Studio interface. The 'File' menu is open, and the 'Open Project...' option is highlighted. The 'Open Project' dialog box is displayed, showing the file explorer view. The 'BN_CIVA' folder is selected, and the file 'BN_CIVA.vbproj' is highlighted. The 'File' menu options include: Open, New Project..., New Web Site..., New Team Project..., New File..., Open Project..., Open Web Site..., Close, Close Solution, Save Selected Items, Save Selected Items As..., Save All, Export Template..., Source Control, Page Setup..., Print..., Account Settings..., Recent Files, Recent Projects and Solutions, and Exit. The 'Open Project' dialog shows the file explorer view with the 'BN_CIVA' folder selected, containing files like 'BN_CIVA.sln' and 'BN_CIVA.vbproj'.

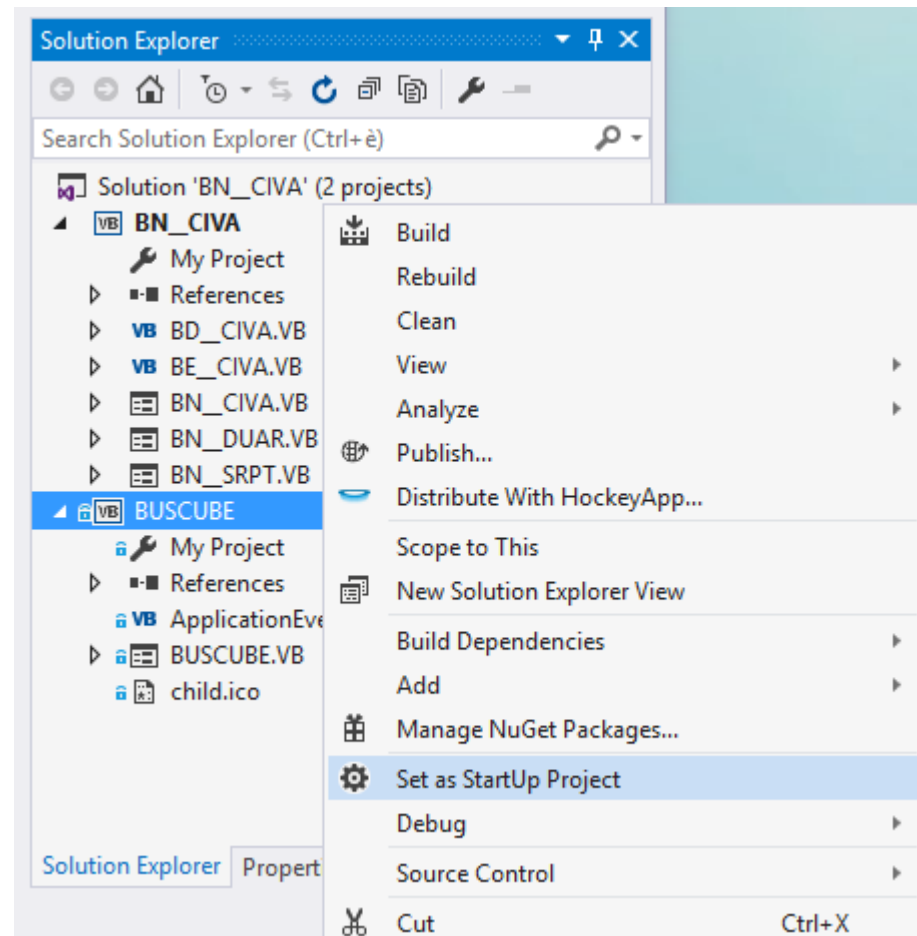
Caricare i progetti in visual studio (e costruire una soluzione)

Aggiungere il progetto Busnet



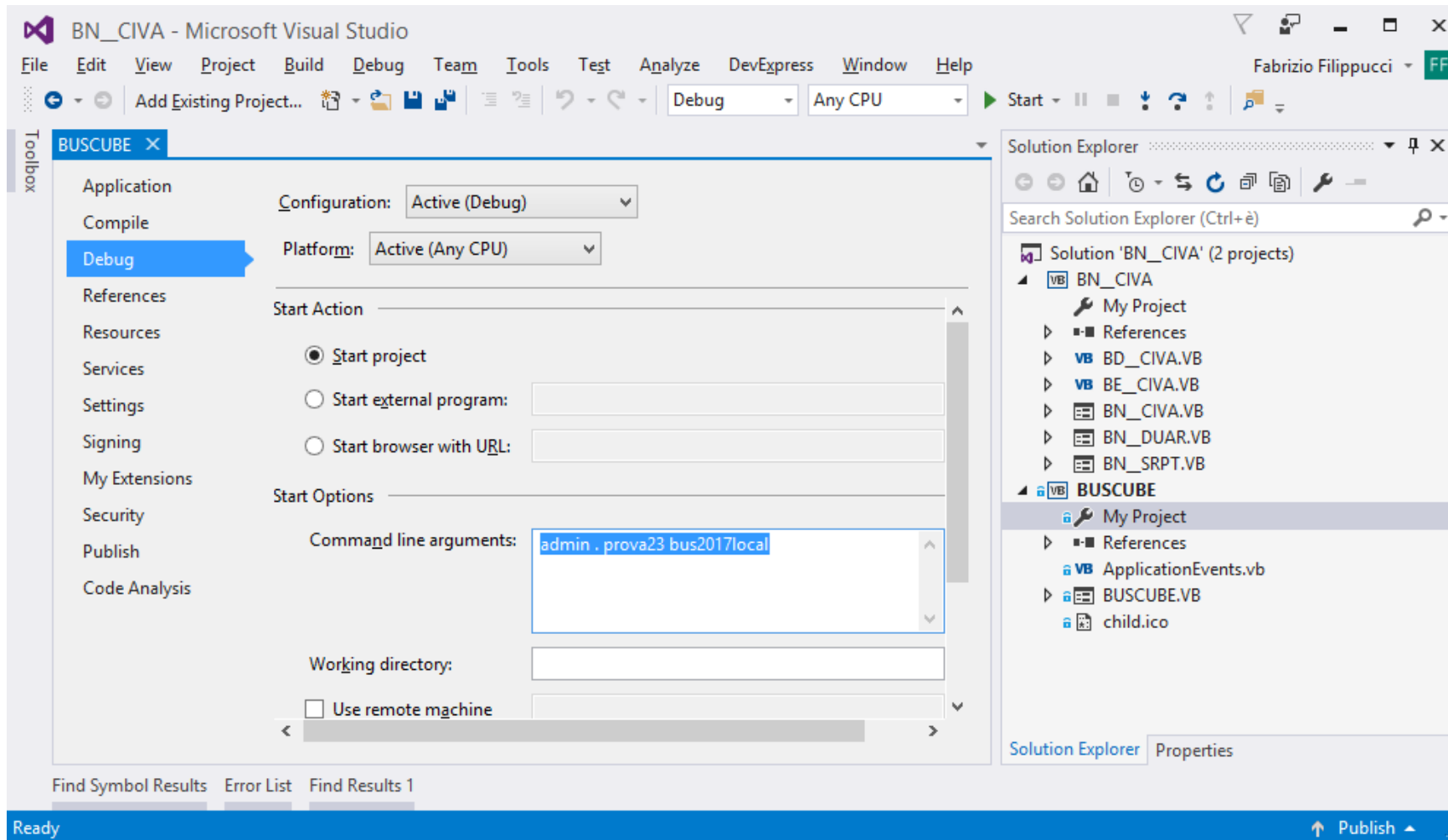
Caricare i progetti in visual studio (e costruire una soluzione)

Impostarlo come progetto di avvio

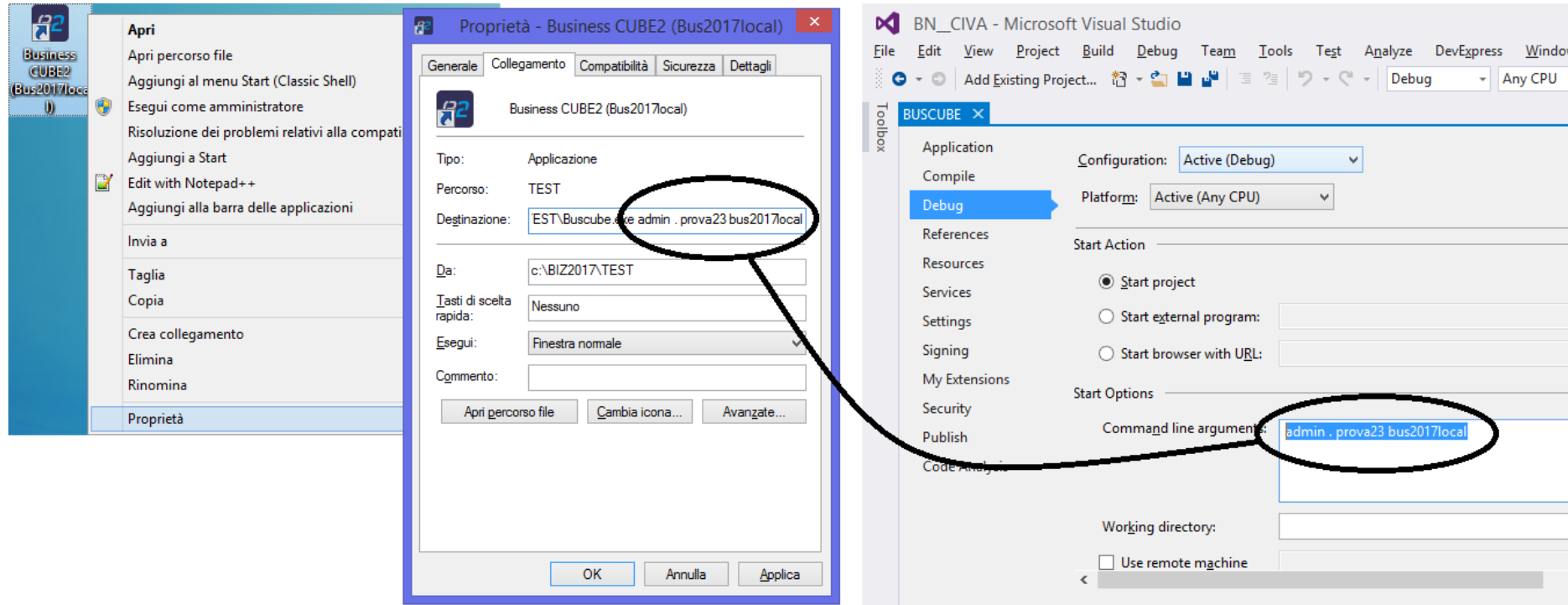


Caricare i progetti in visual studio (e costruire una soluzione)

Impostare eventuali parametri di avvio se necessario



NB: parametri di avvio vanno impostati come l'icona presente su desktop



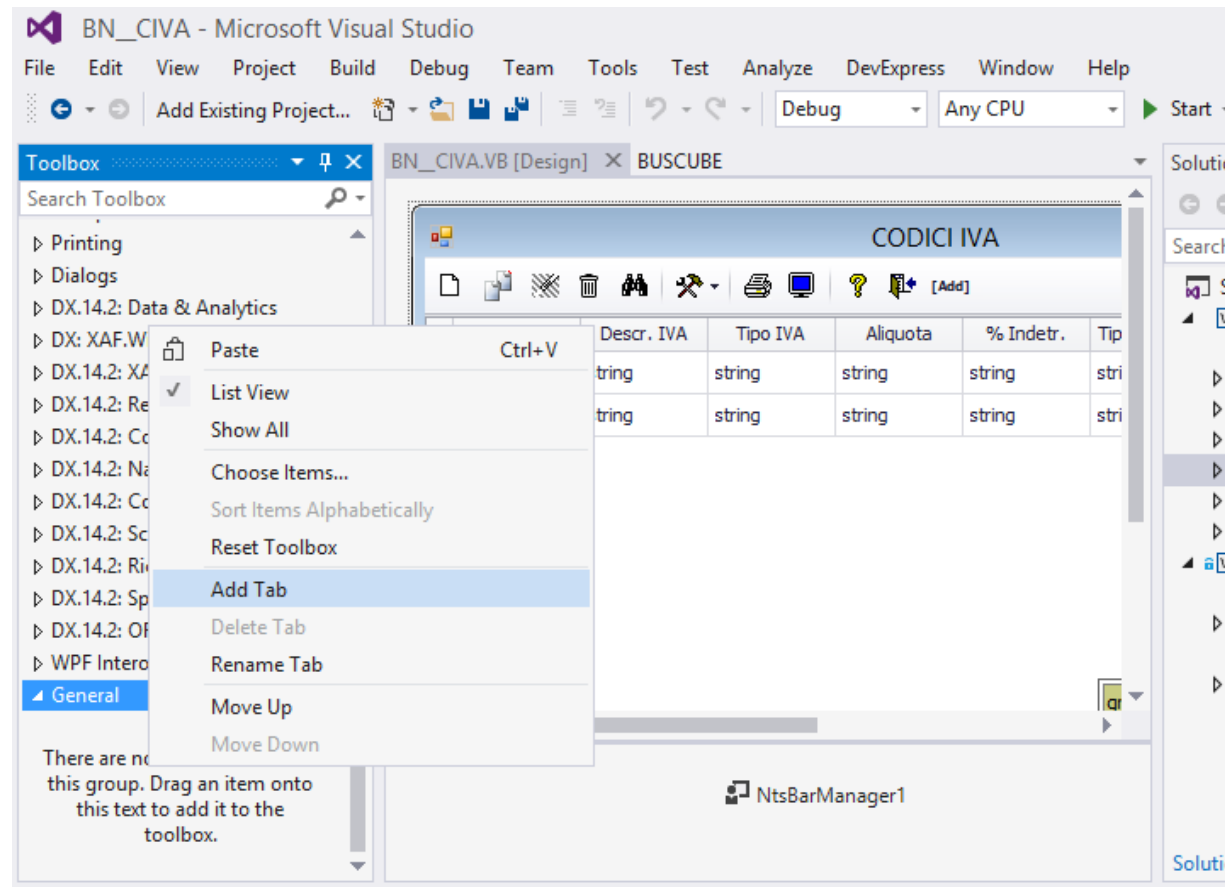
Caricare i progetti in visual studio (e costruire una soluzione)

- Salvare la soluzione e riaprirla
- Tasti 'mostra tutti i file', 'visualizza codice' e 'visualizza finestra di progettazione'
- All'avvio della progettazione di una UI chiudere la modale della DevExpress



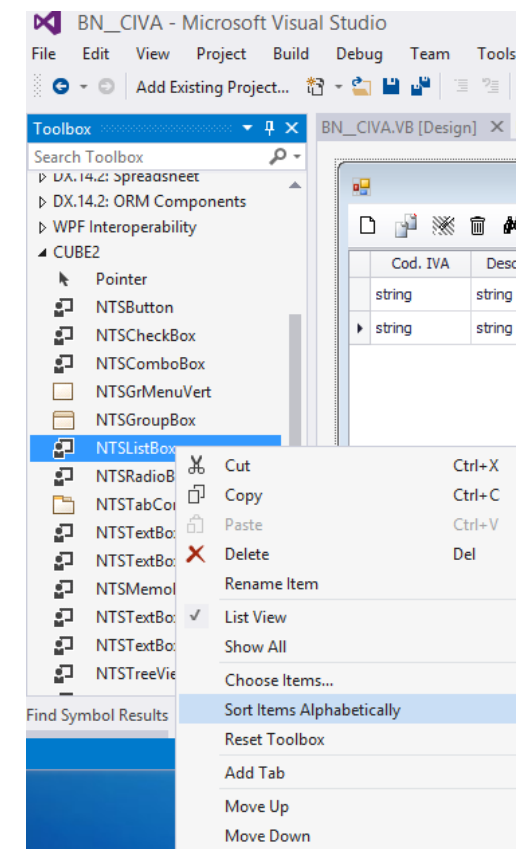
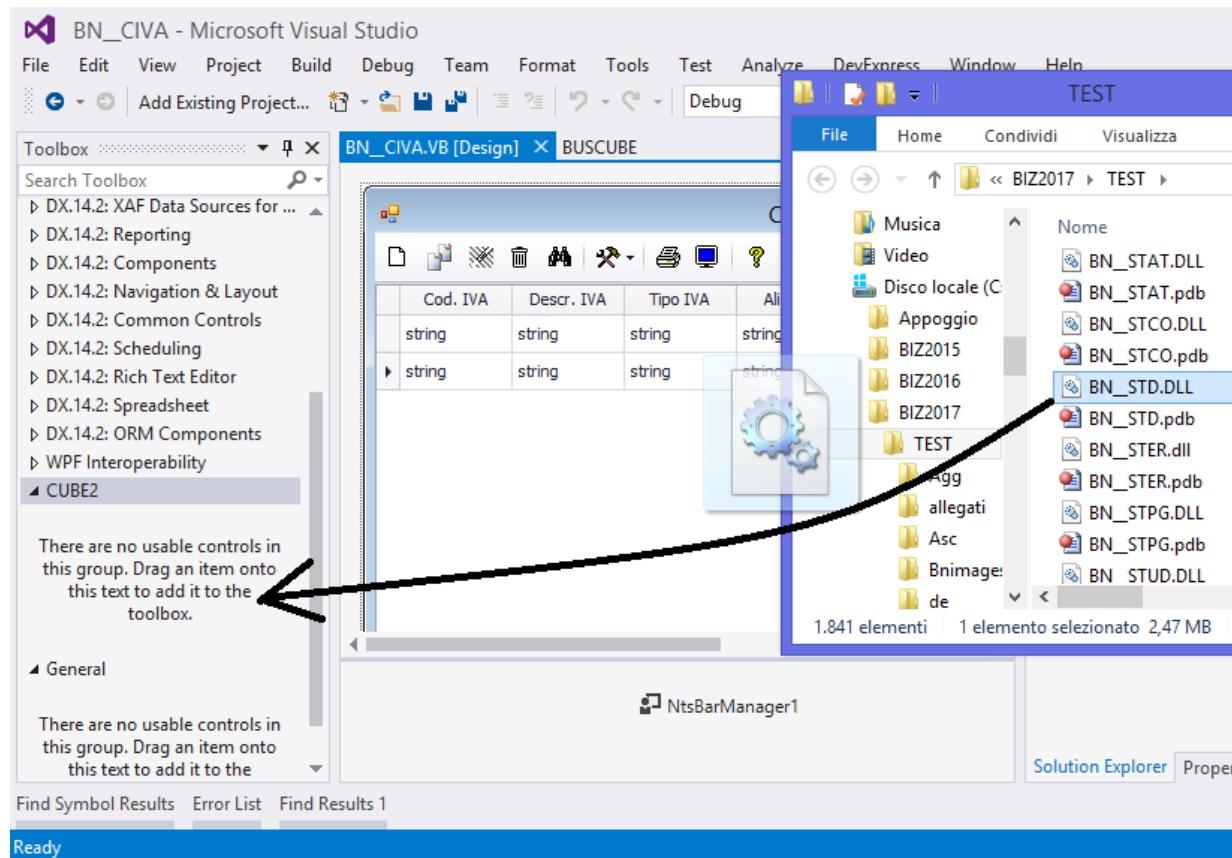
Caricare i progetti in visual studio (e costruire una soluzione)

- Nella casella degli strumenti aggiungere una scheda per i controlli Cube 2

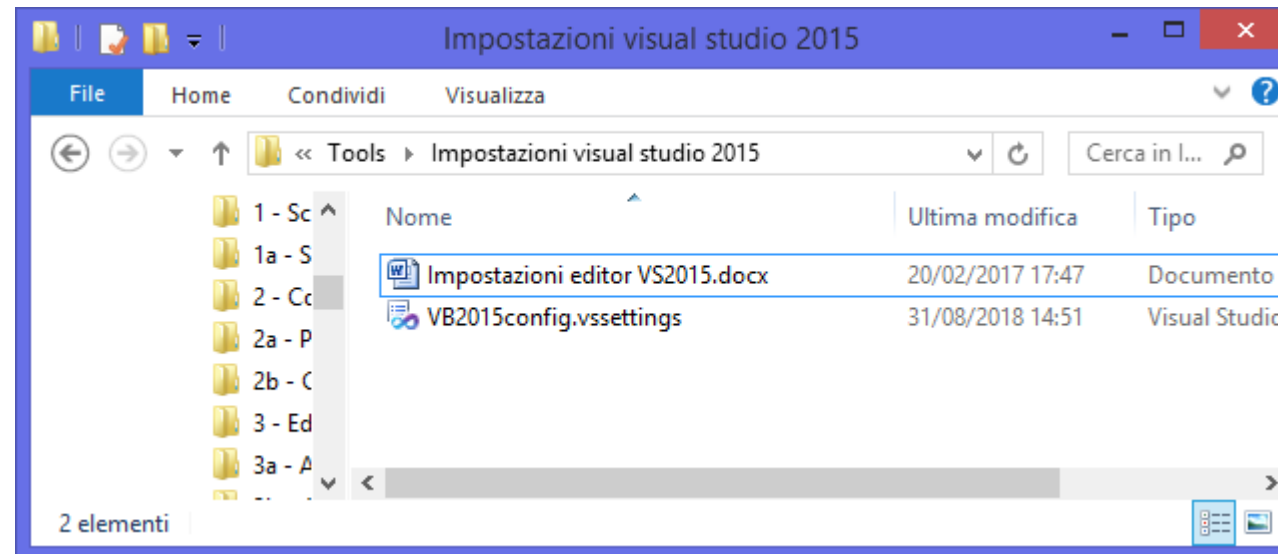


Caricare i progetti in visual studio (e costruire una soluzione)

- Trascinare la BN__STD.DLL presente nella BIZ20xx\TEST nella nuova scheda



- Opzioni Strict On
-(per chi vuole importare le configurazioni di Nts -> Tools\Impostazioni editor vb.net 2015)



Perché non si ferma in DEBUG ? (possibili cause/soluzioni)

The screenshot shows the Microsoft Visual Studio IDE with the 'BN_CIVA' project open. The 'Compile' button in the toolbar is highlighted with a red arrow. Another red arrow points from the 'Advanced Compile Options' button in the 'Advanced Compile Options' dialog to the 'Advanced Compiler Settings' dialog. The 'Advanced Compiler Settings' dialog shows the 'Optimizations' tab with 'Generate debug info' set to 'Full' and 'Define DEBUG constant' checked.

Advanced Compile Options

Configuration: Active (Debug) Platform: Active (Any CPU)

Target CPU: AnyCPU

☐ Prefer 32-bit

Warning configurations:

Condition	Notification
Implicit conversion	
Late binding; call could fail at run time	
Implicit type; object assumed	
Use of variable prior to assignment	

☐ Disable all warnings

☐ Treat all warnings as errors

☐ Generate XML documentation file

☐ Register for COM interop

Advanced Compile Options...

Advanced Compiler Settings

Optimizations

☐ Remove integer overflow checks ☐ Enable optimizations

DLL base address: &H00400000

Generate debug info: Full

Compilation Constants

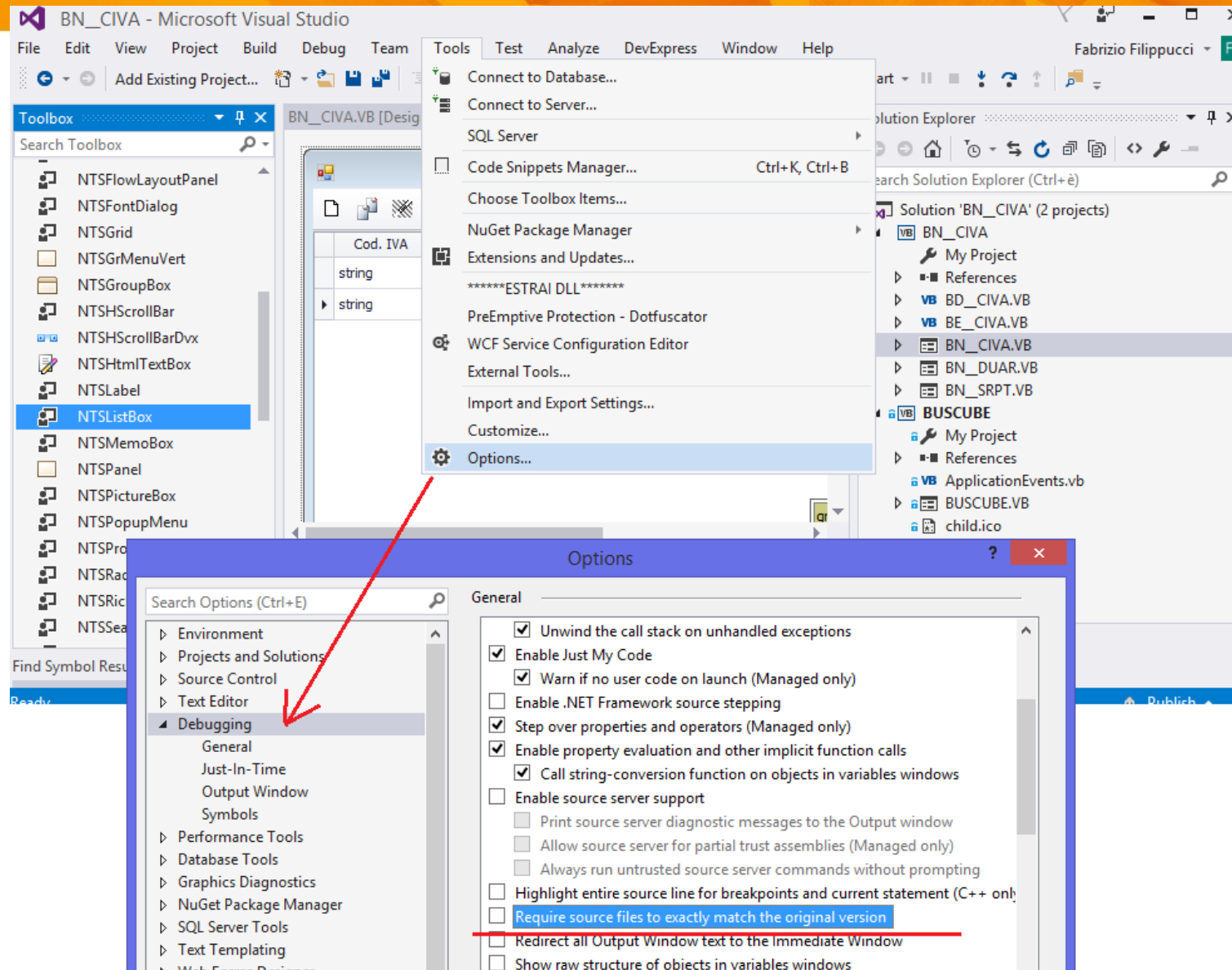
☒ Define DEBUG constant ☒ Define TRACE constant

Custom constants:

Example: Name1="Value1",Name2="Value2",Name3="Value3"

Generate serialization assemblies:

Perché non si ferma in DEBUG? (possibili cause/soluzioni)



Caratteristiche di un reference (regola)

The screenshot shows the Visual Studio interface with two windows open: Solution Explorer and Properties.

Solution Explorer: Displays the project structure for 'Solution 'BN_CIVA' (2 projects)'. Under the 'References' folder, 'BN_STD' is selected. The list of references includes:

- DevExpress.Data.v14.2
- DevExpress.Utils.v14.2
- DevExpress.XtraBars.v14.2
- DevExpress.XtraEditors.v14.2
- DevExpress.XtraGrid.v14.2
- System
- System.Data
- System.Deployment
- System.Drawing
- System.Runtime.Remoting
- System.Windows.Forms
- System.Xml
- VB BD_CIVA.VB
- VB BE_CIVA.VB
- BN_CIVA.VB
- BN_DUAR.VB

Properties Window: Displays the 'BN_STD Reference Properties'. The following properties are circled in red:

- (Name): BN_STD
- Copy Local: False
- Specific Version: False

The full list of properties in the window is:

Property	Value
(Name)	BN_STD
Copy Local	False
Culture	
Description	
Embed Interop Types	False
File Type	Assembly
Identity	BN_STD
Path	C:\BIZ2017\TEST\BN_STD.dll
Resolved	True
Runtime Version	v4.0.30319
Specific Version	False
Strong Name	False
Version	23.0.1.0

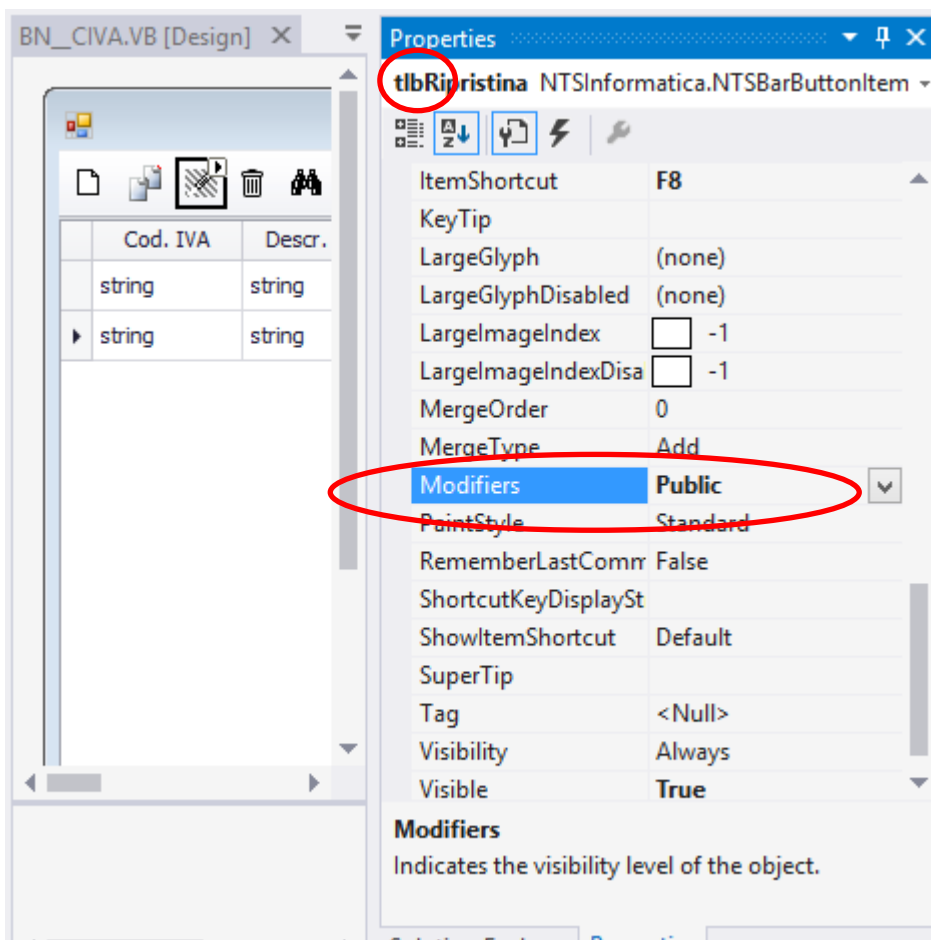
Copy Local

Indicates whether the reference will be copied to the output directory.

Specific Version

Indicates whether this reference is to a specific version of an assembly.

- Visibilità = Public
- Prefisso

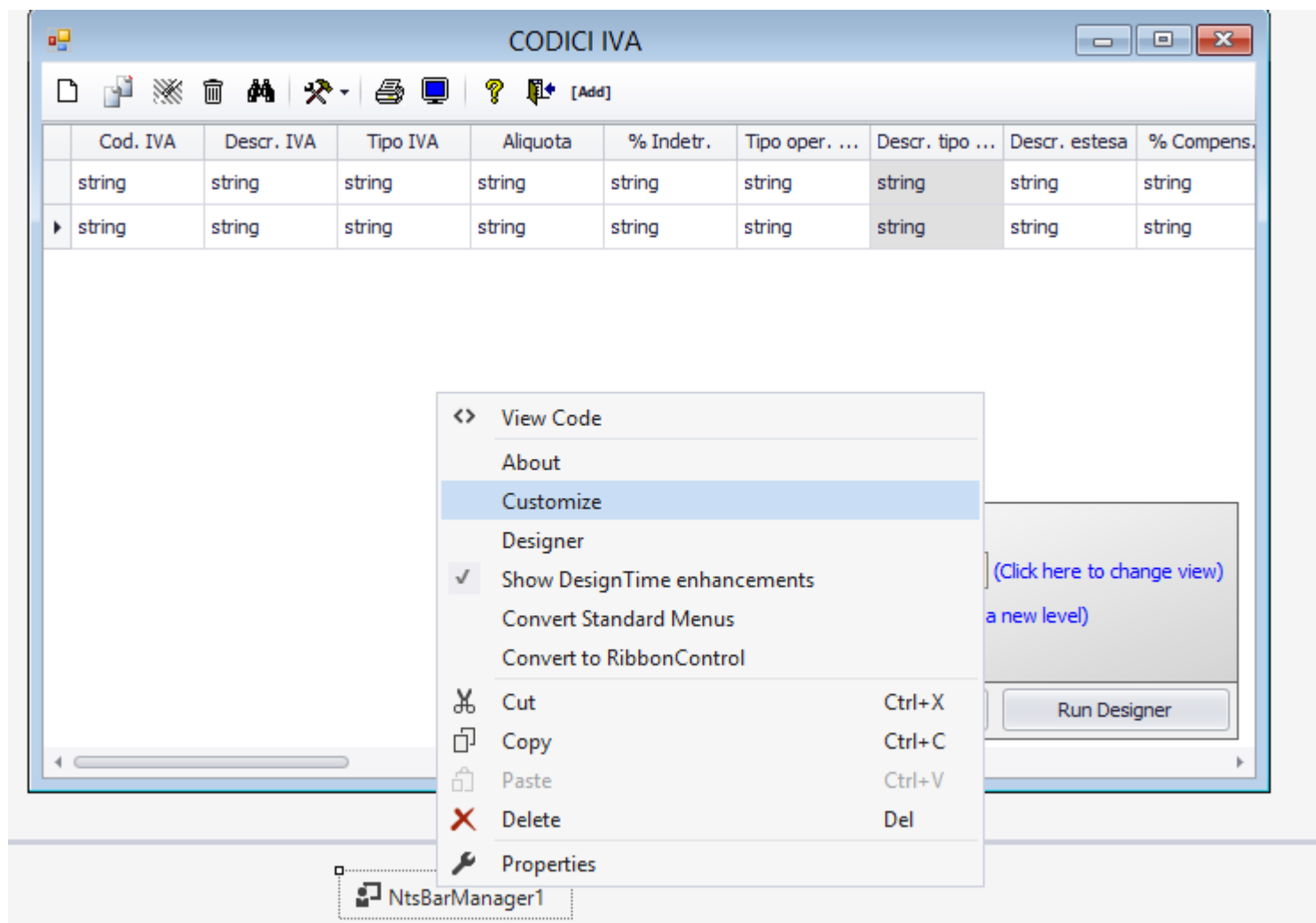


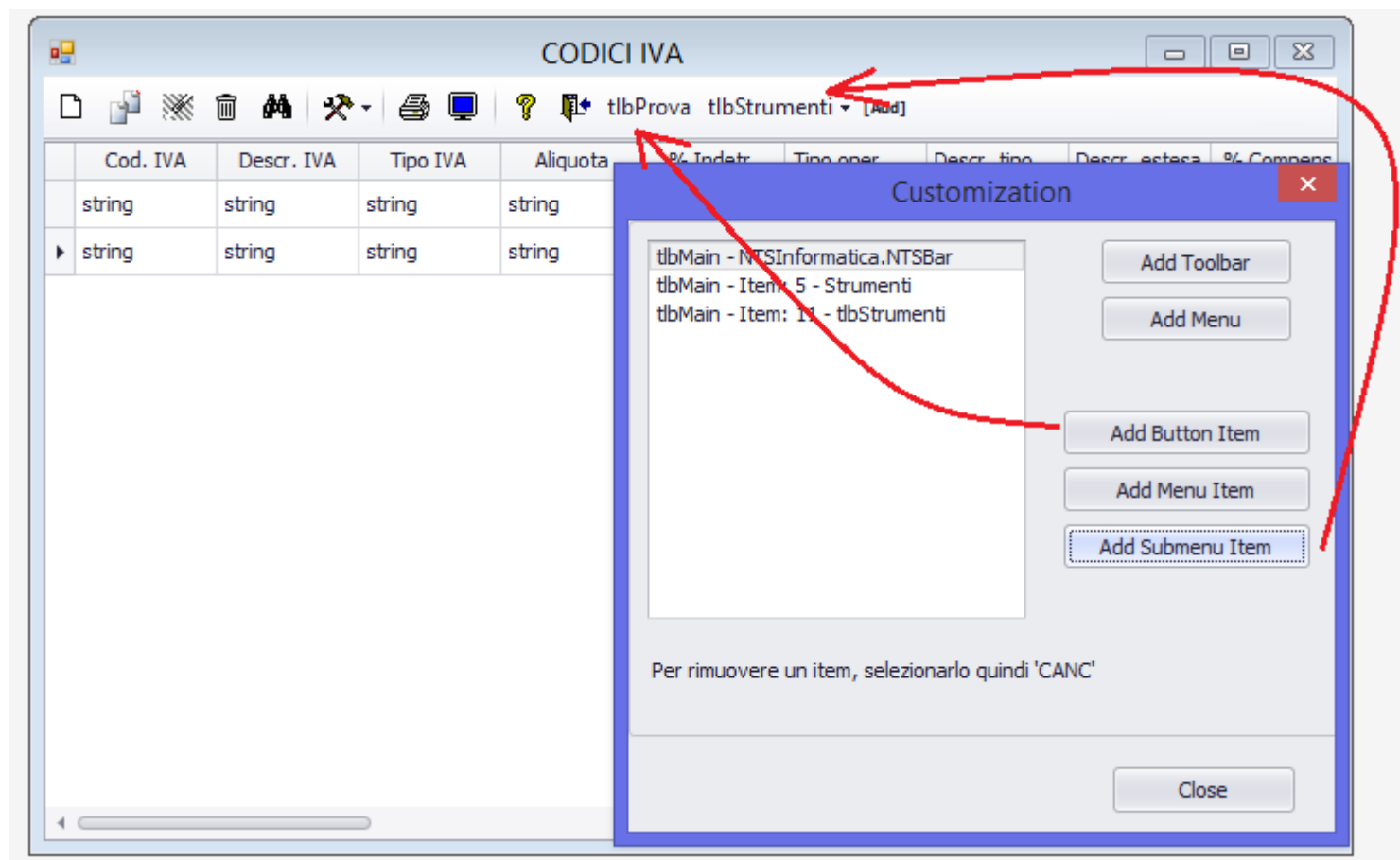
Tipo controllo	Prefisso
NTSPanel	pn
NTSBar, NTSBarButtonItem, NTSBarSubItem	tlb
NTSGrid	gr
NTSGridView	grv
NTSTextBoxNum, NTSTextBoxData, NTSTextBoxStr	ed
NTSLabel	lb
NTSComboBox	cb
NTSCheckBox	ck
NTSGroupBox	fm
NTSButton	cmd
NTSRadioButton	op
NTSTabPage	tbp

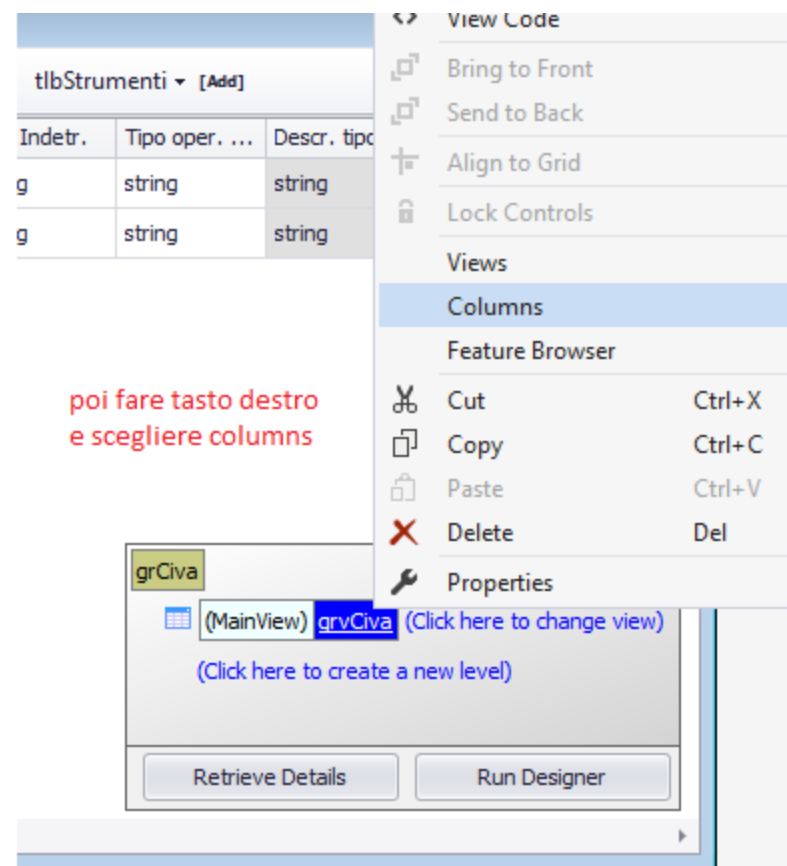
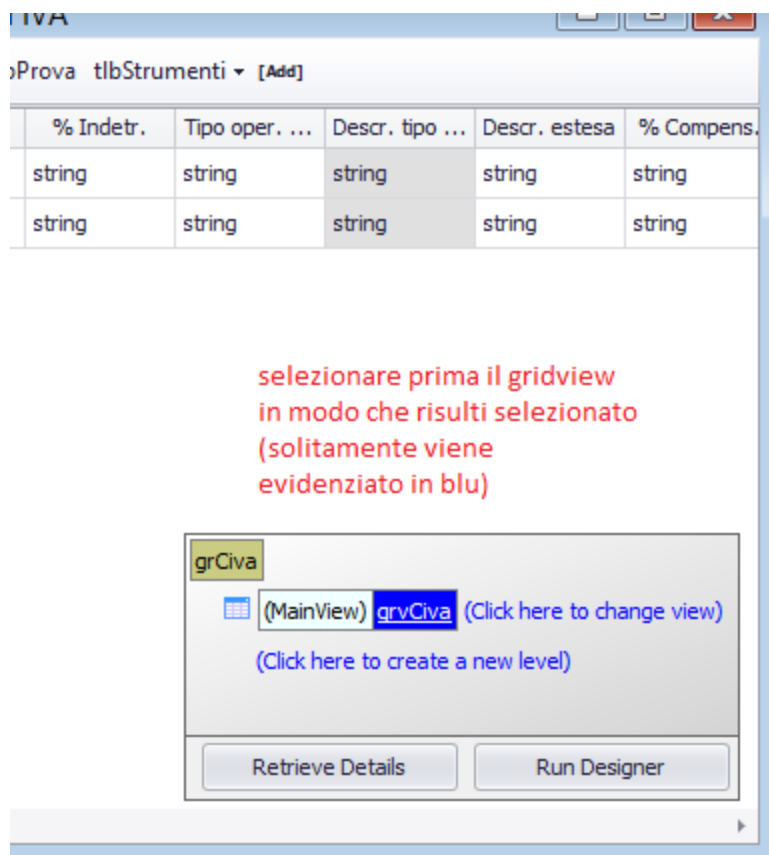
- Visibilità = Public
- Prefisso

Tipo oggetto \ variabile	Prefisso
UI	frm
Entity	oCle
DAL	oCId
Oggetto generico	o
Boolean	b
String	str
Integer	l
Decimal	d
Date	dt

Aggiungere pulsanti e sottomenu in toolbar



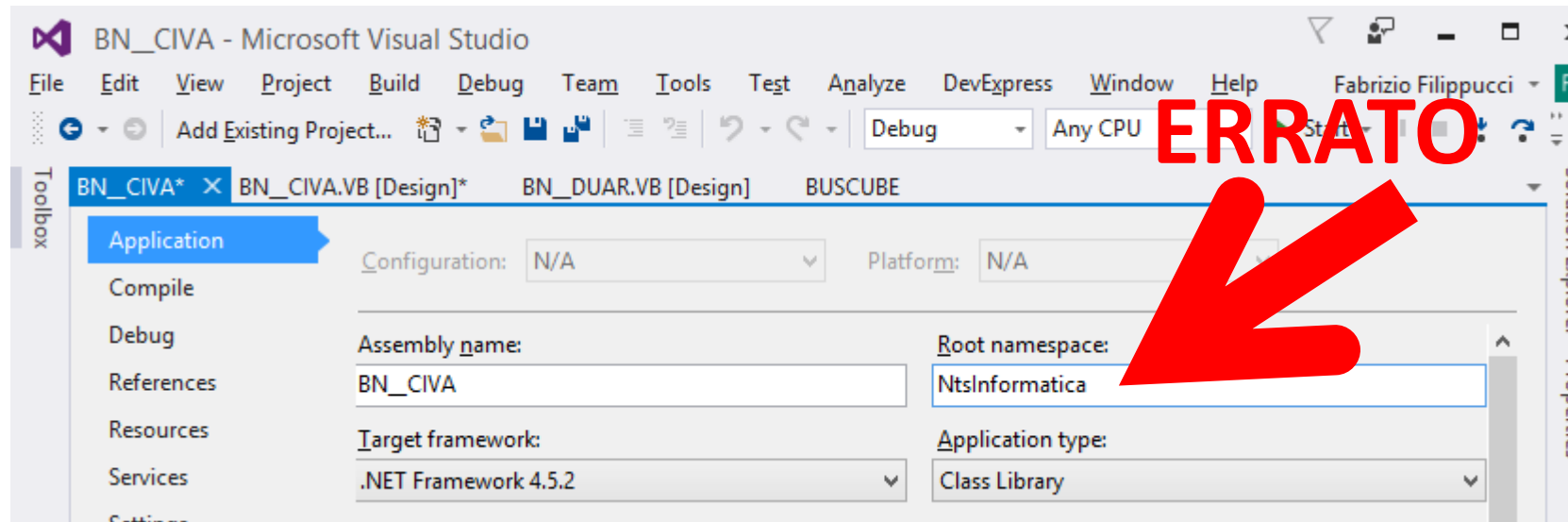


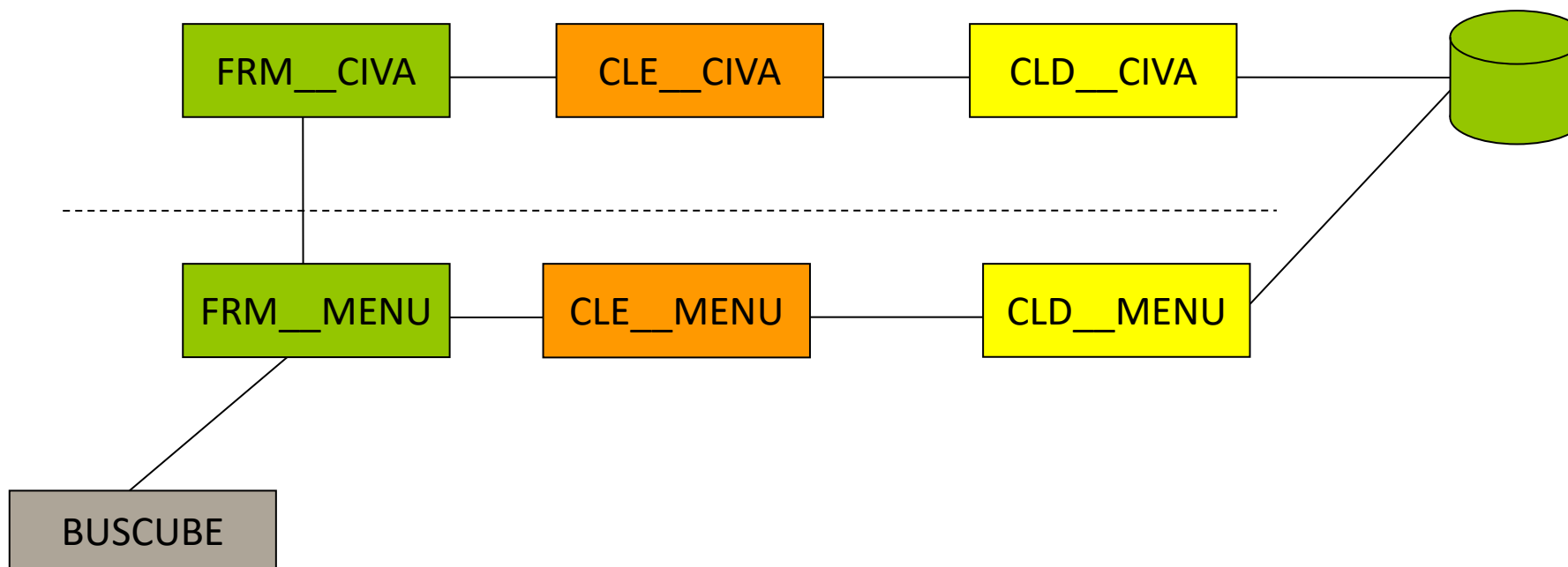


NB: il nomi sono CASE-SENSITIVE!!!!!!!!!!

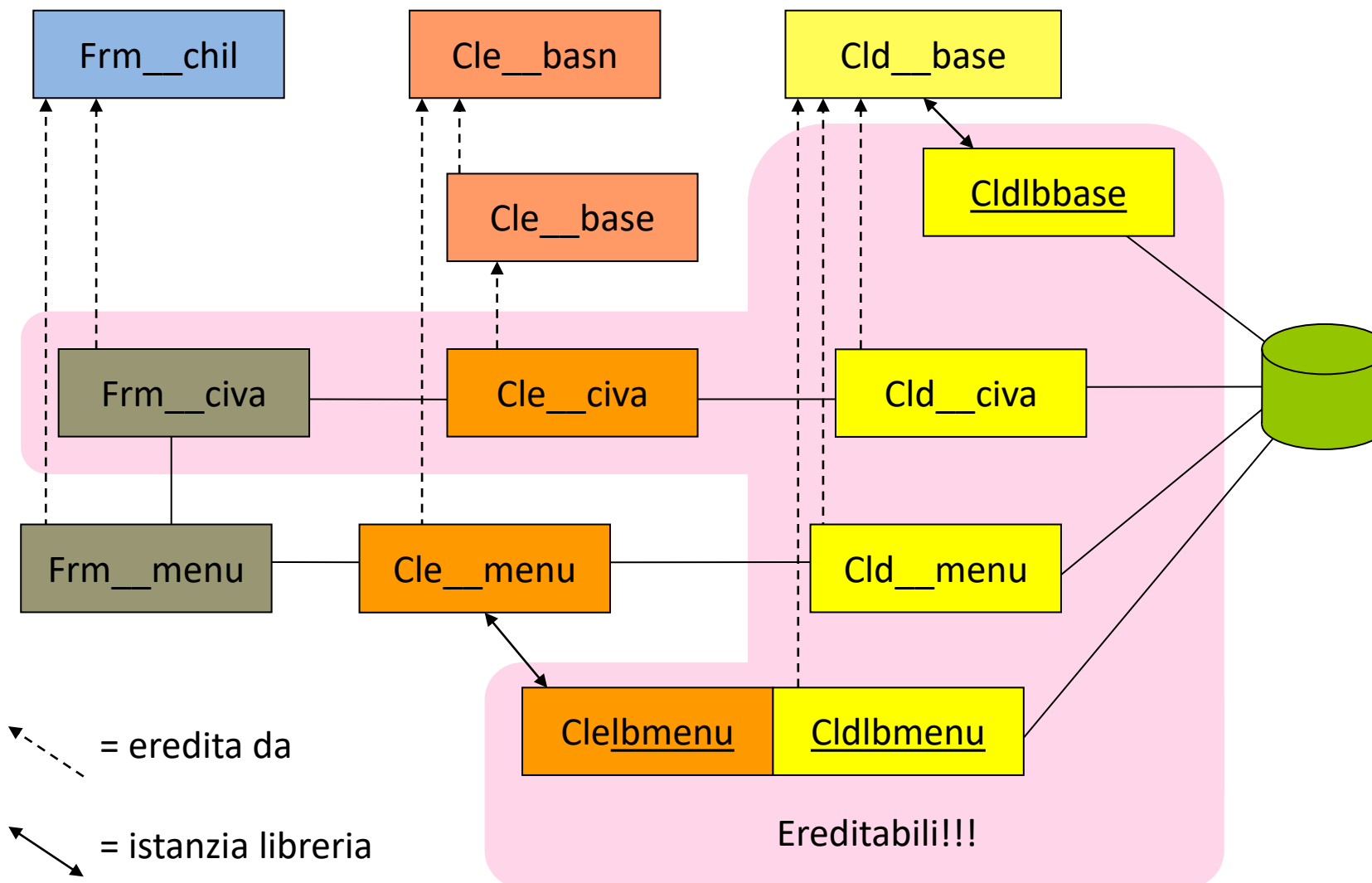
Assembly name : TUTTO MAIUSCOLO

Root namespace : NTSInformatica ('NTSI' maiuscolo ed il resto minuscolo)





Istanze e proprietari istanze



Classi base (ereditate dai 3 livelli)

- FRM__CHIL classe base per UI
- CLE__BASN classe base per entity
- CLE__BASE classe base per entity (opzionale)
- CLD__BASE classe base per DAL

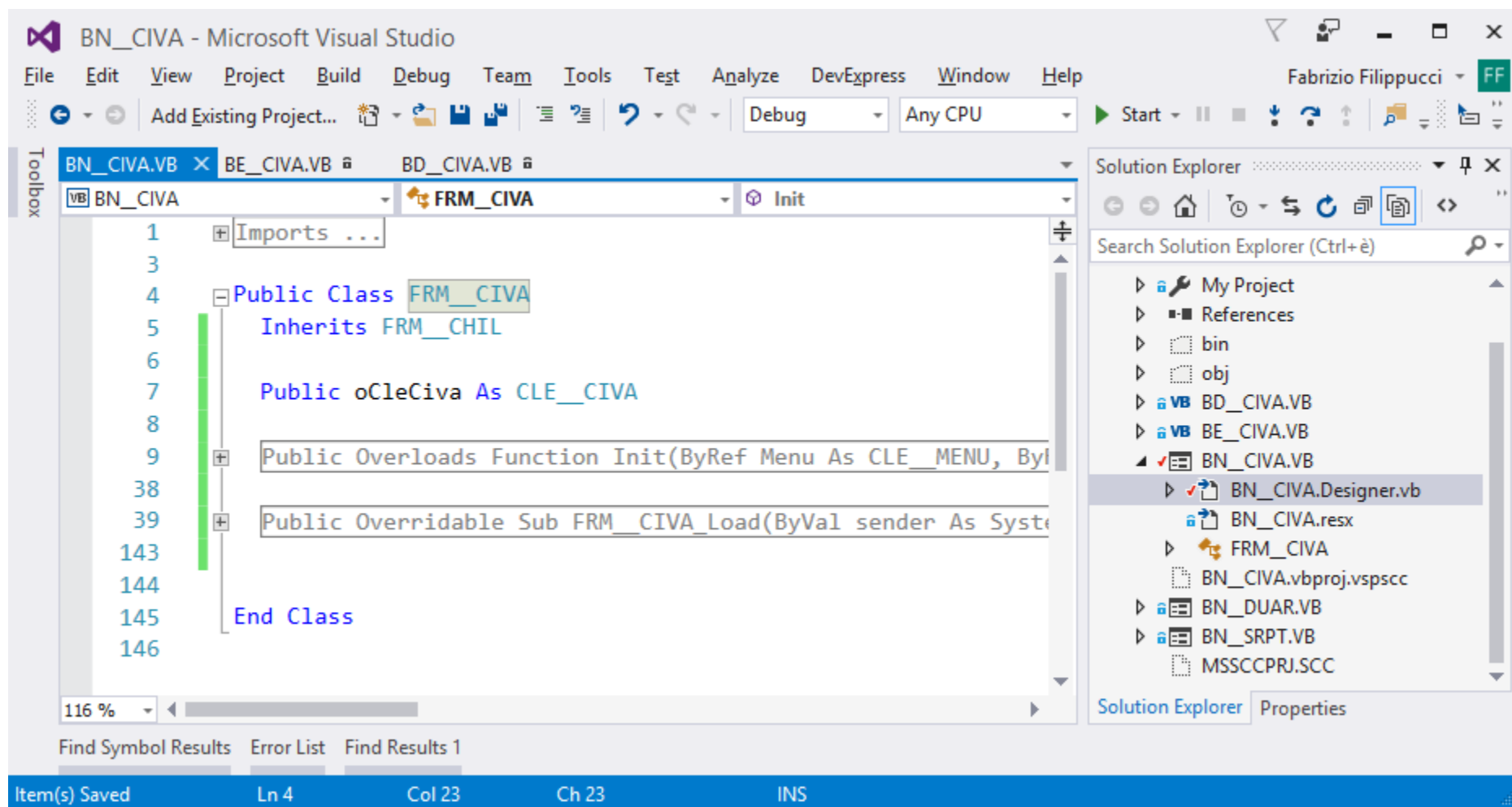
Librerie

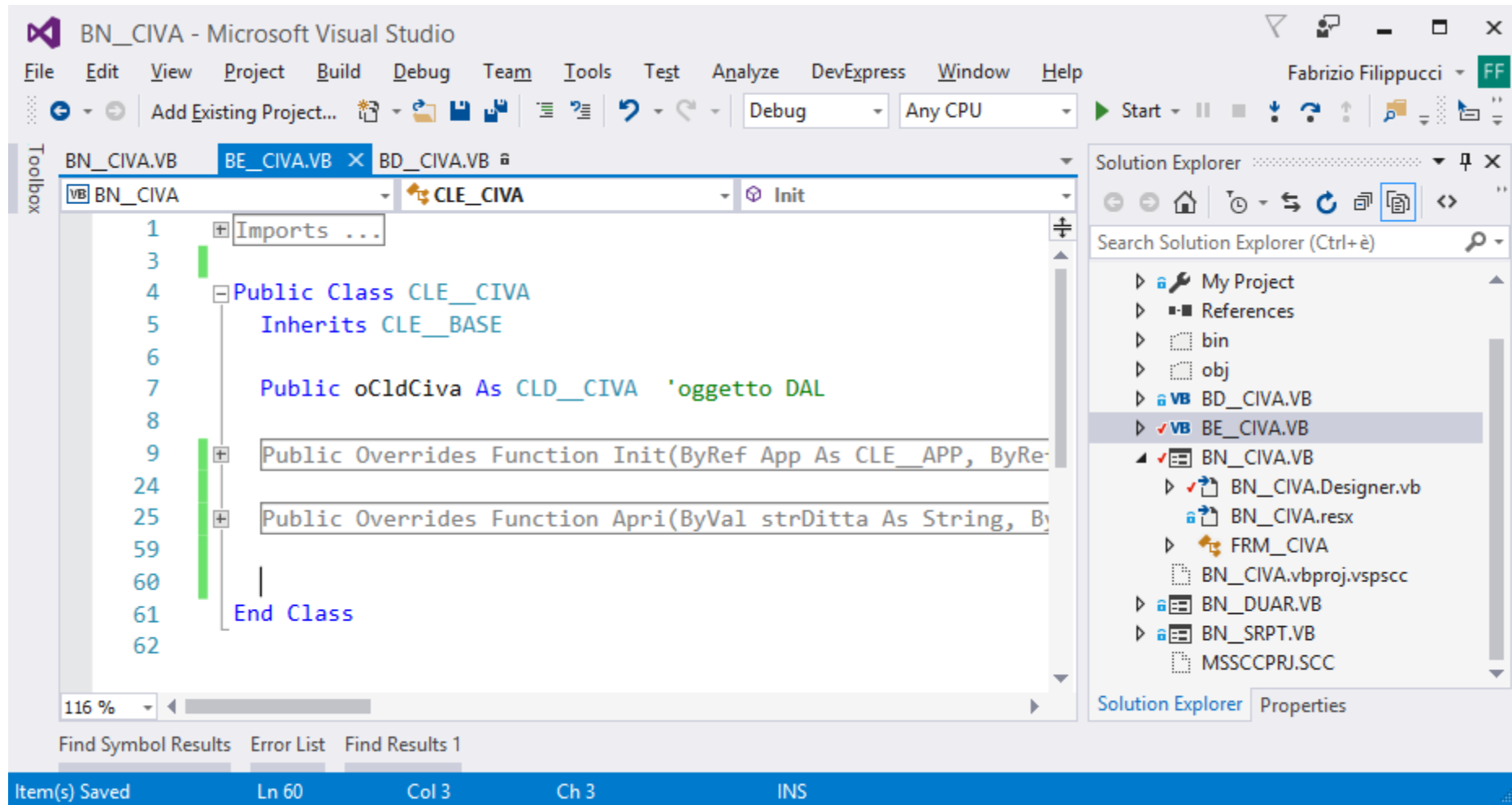
- CLELBMENU, CLDLBMENU libreria funzioni comuni modificabili
- CLDLBBASE libreria funzioni comuni modificabili

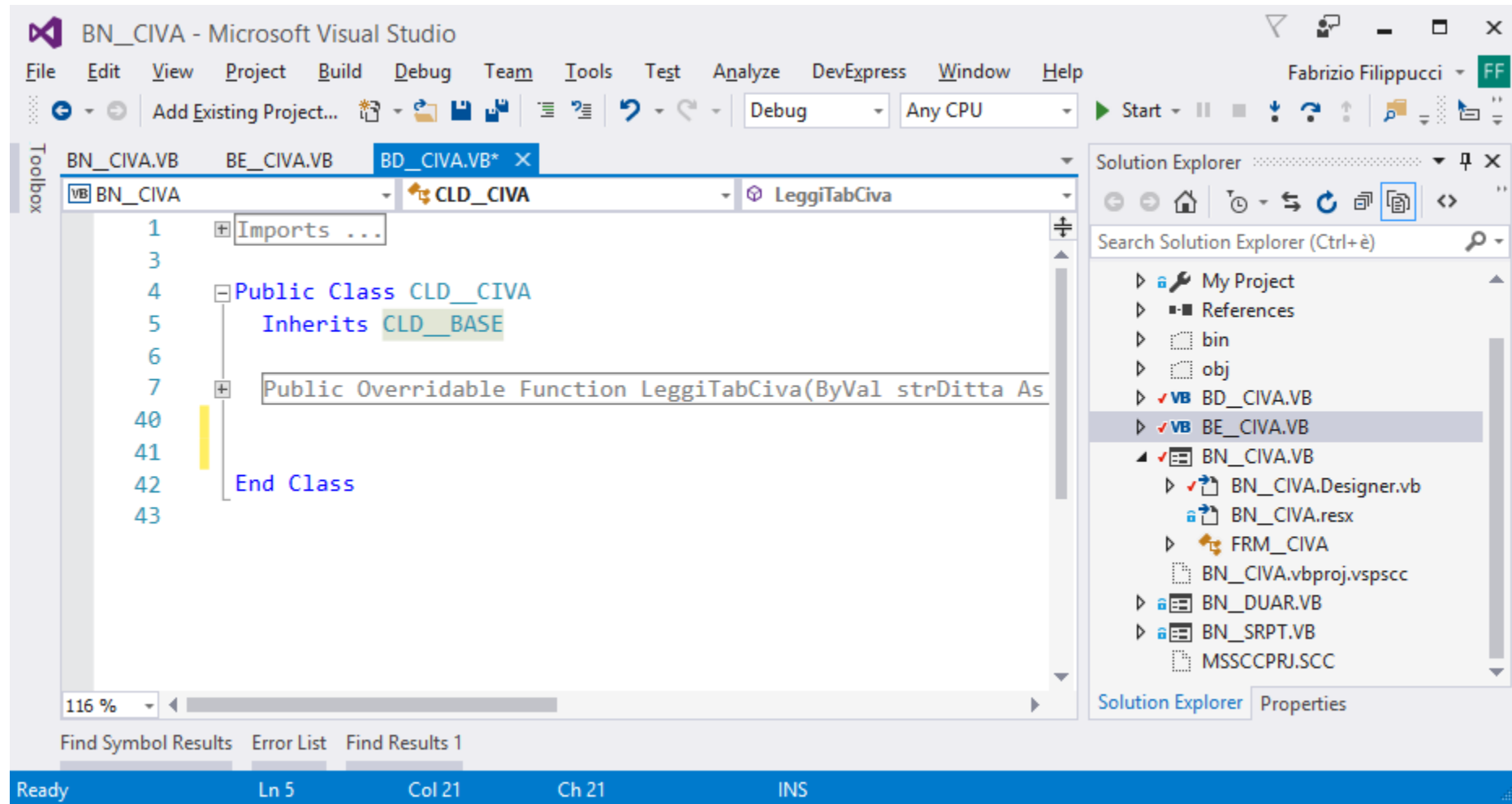
Uno sguardo preliminare ad un semplice child (eventi)

BN__CIVA.FRM__CIVA

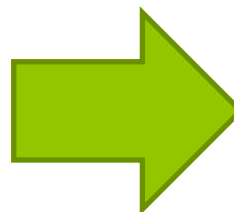
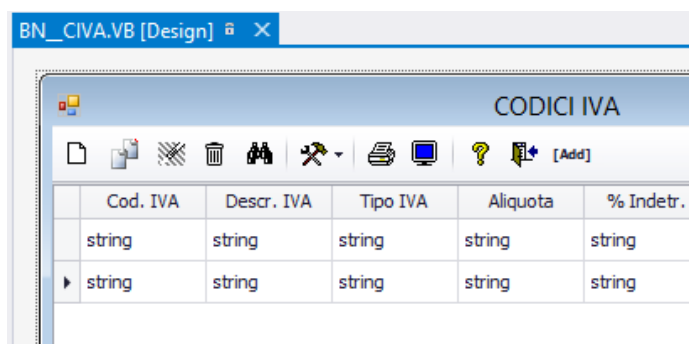
- Intit
 - InitalizeComponent
 - InstanziaDLL → BN__CIVA.CLE__CIVA
- Form_LOAD
 - Initcontrols [CaricaCombo]
 - Apri(dsCiva) → BN__CIVA.CLD__CIVA
 - Intit
 - ✓ InstanziaDLL →
 - Leggi
 - Leggi(dsShared)
 - BindingControls (dcCiva)





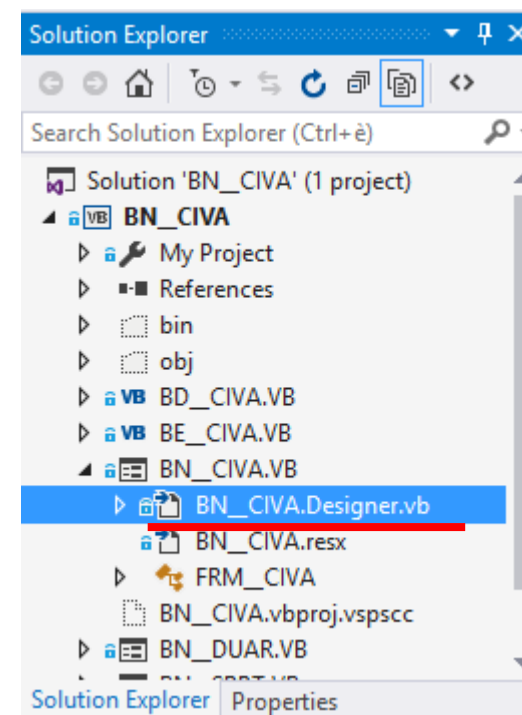


- E' il designer che la popola



```
Public Overridable Sub InitializeComponent() ...
```

- Il livello UI ha sempre un designer collegato (Partial Class)



```
Public Overloads Function Init(ByRef Menu As CLE__MENU, ByRef Param As CLE__CLDP, Optional ByVal Ditta As String = "",
                              Optional ByRef SharedControls As CLE__EVNT = Nothing) As Boolean

    oMenu = Menu
    oApp = oMenu.App
    oCallParams = Param
    If Ditta <> "" Then
        DittaCorrente = Ditta
    Else
        DittaCorrente = oApp.Ditta
    End If

    InitializeComponent()
    Me.MinimumSize = Me.Size

    '-----
    'Creo e attivo l'entity, in più inizializzo la funzione che dovrà rilevare gli eventi dall'ENTITY stesso
    Dim strErr As String = ""
    Dim objTmp As Object = Nothing
    If CLN__STD.NTSIstanziaDll(oApp.ServerDir, oApp.NetDir, "BN__CIVA", "BE__CIVA", objTmp, strErr, False, "", "") = False Then
        oApp.MsgBoxErr(oApp.Tr(Me, 128182574024687500, "ERRORE in fase di creazione Entity:" & vbCrLf & "|" & strErr & "|"))
        Return False
    End If
    oCleCiva = CType(objTmp, CLE__CIVA)
    '-----

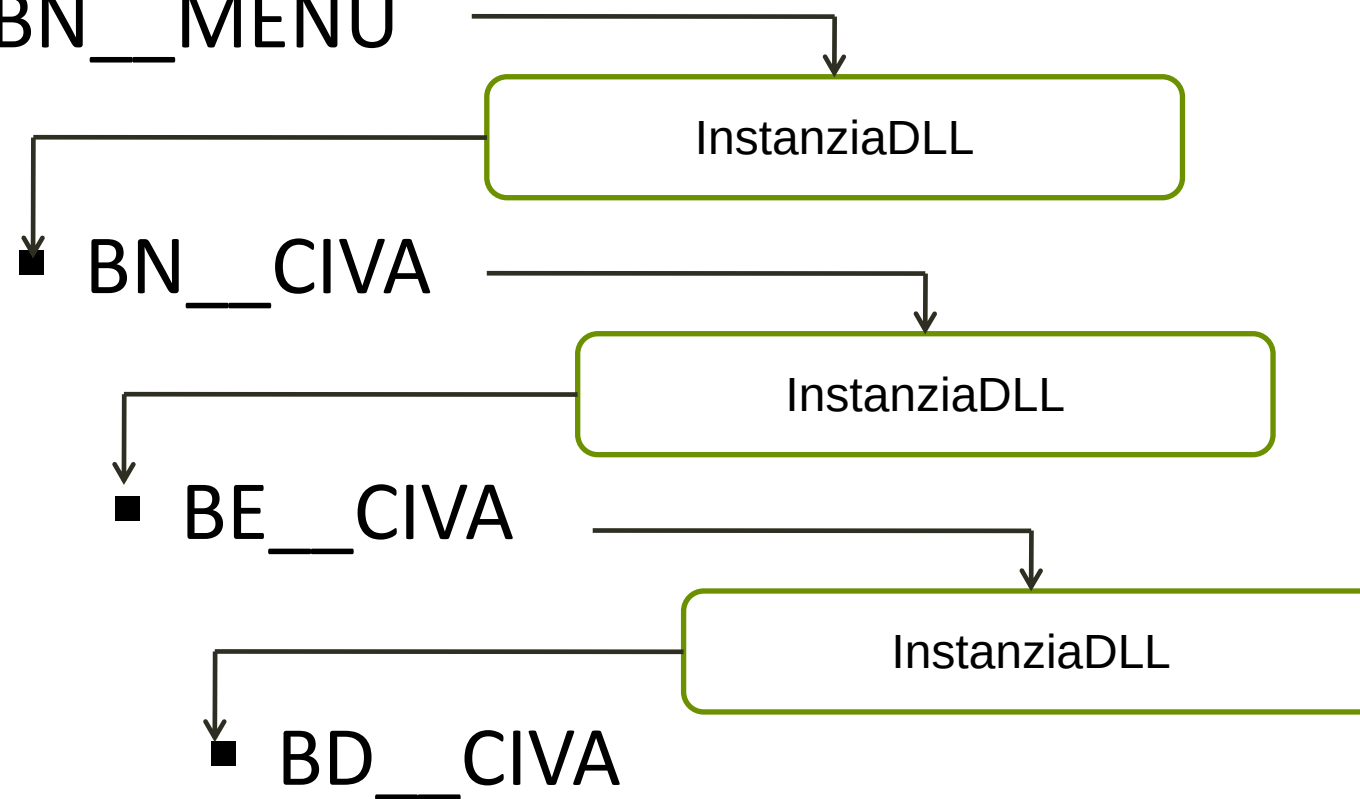
    AddHandler oCleCiva.RemoteEvent, AddressOf GestisciEventiEntity
    If oCleCiva.Init(oApp, NTSScript, oMenu.oCleComm, "TABCIVA", False, "", "") = False Then Return False

    Return True
End Function
```

Uno sguardo preliminare ad un semplice child (eventi)

- Cosa succede?

- BN__MENU



Questo permette l'ereditarietà!

Uno sguardo preliminare ad un semplice child (oggetti, variabili)

FRM

oCallParams (CLE__CLDP)

oMenu (CLE__MENU)

oApp (CLE__APP)

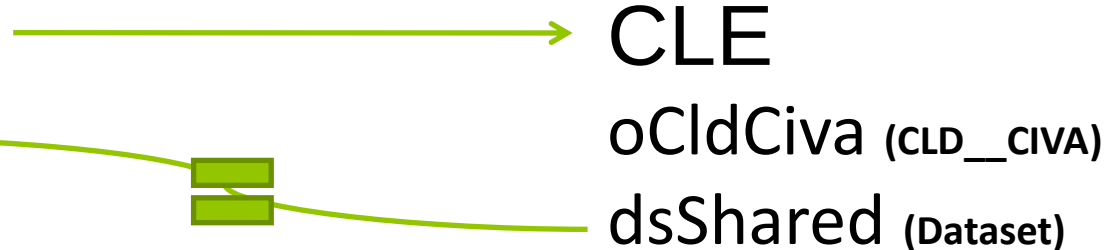
DittaCorrente (String)

oCleCiva (CLE__CIVA)

dsCiva (Dataset)

dcCiva (BindingSource)

oMenu.oCleComm (BELBMENU)



Dataset, Datatable, Datarow, DataColumn (alcune note)

Dataset

Datatable 1		

Datarow

...

Datatable N			

DataColumn

Dataset, Datatable, Datarow, DataColumn (alcune note)

Definizione:

```
Dim dsNuovoDataSet As DataSet
```

```
Dim dttNuovoDataTable As DataTable
```

```
Dim dtrNuovoDataRow As DataRow
```

```
Dim dtcNuovoDataColumn As DataColumn
```

Esempio:

```
Dim dsProva As New DataSet
```

```
Dim dttTabella As New DataTable("MIAPRIMA")
```

```
dsProva.Tables.Add(dttTabella)
```

```
dttTabella.Columns.Add(New DataColumn("xx_vallotto", GetType(Decimal)))
```

```
dttTabella.Columns.Add(New DataColumn("xx_clfor", GetType(String)))
```

```
dttTabella.Columns.Add(New DataColumn("xx_datcarico", GetType(Date)))
```

```
dttTabella.Columns.Add("xx_numero", GetType(Integer))
```

```
dttTabella.Rows.Add(New Object() {1.5, "uno punto cinque", Now, 10})
```

```
dttTabella.Rows.Add(New Object() {2, "due", Now, 10})
```

```
dttTabella.Rows.Add(New Object() {31.5, "prova", Now, 10})
```

```
dttTabella.Rows.Add(New Object() {4.5, "xxx", NTSCDate("15/11/2012"), 10})
```

Definizione:

```
Dim dsNuovoDataSet As DataSet  
Dim dttNuovoDataTable As DataTable  
Dim dtrNuovoDataRow As DataRow  
Dim dtcNuovoDataColumn As DataColumn
```

Esempio:

```
Dim dsProva As New DataSet  
Dim dttTabella As New DataTable("MIAPRIMA")  
dsProva.Tables.Add(dttTabella)  
dttTabella.Columns.Add(New DataColumn("xx_vallotto", GetType(Decimal)))  
dttTabella.Columns.Add(New DataColumn("xx_clfor", GetType(String)))  
dttTabella.Columns.Add(New DataColumn("xx_datcarico", GetType(Date)))  
dttTabella.Columns.Add("xx_numero", GetType(Integer))  
dttTabella.Rows.Add(New Object() {1.5, "uno punto cinque", Now, 10})  
dttTabella.Rows.Add(New Object() {2, "due", Now, 10})  
dttTabella.Rows.Add(New Object() {31.5, "prova", Now, 10})  
dttTabella.Rows.Add(New Object() {4.5, "xxx", NTSCDate("15/11/2012"), 10})  
dsProva.AcceptChanges()
```


Accedere ad un dato:

```
If NTSCDec(dttTabella.Rows(0).Item("xx_cifra")) = 1.5 Then
```

```
    oApp.MsgBoxInfo("OK!")
```

```
End If
```

'Altri esempi

```
'dttTabella.Rows(0)!xx_cifra
```

```
'dsProva.Tables("MIAPRIMA").Rows(0)!xx_cifra
```

Scan della struttura:

```
For i As Integer = 0 To dttTabella.Rows.Count - 1
```

```
    dttTabella.Rows(i).Item("xx_numero") = 1
```

```
Next
```

'oppure

```
For Each dtrTmp As DataRow In dttTabella.Rows
```

```
    dtrTmp!xx_numero = 1
```

'oppure

```
'dtrTmp.Item("xx_numero") = 1
```

```
Next
```

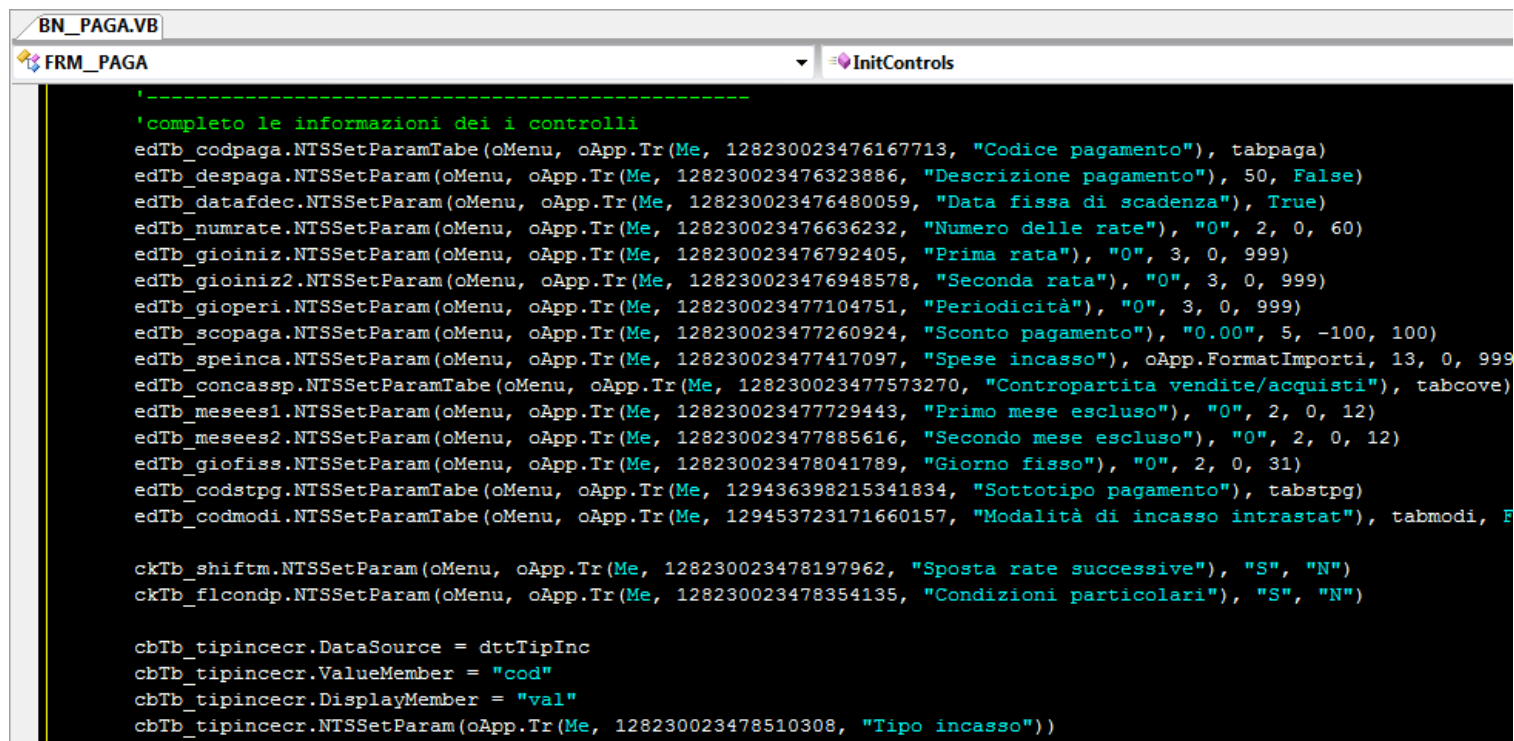
Praticamente è la “InitializeComponent” di Nts:

- InitControlsBeginInit(Me, False)
 - inibisce gli eventi dei controlli UI
- Carico le immagini della Toolbar
 - imposta le immagini sulla toolbar prendendole dalla BNIMAGES
- [CaricaCombo]
 - definisce le voci da visualizzare sui combo
- .NTSSetParam(
 - permette di definire il nome del campo, formato, lunghezza, minimo, massimo, ...
- .NTSSetParamTab(
 - come la precedente ma le informazioni le preleva dalla BN__STD.nome_tabella
- .NTSSetParamZoom(
 - permette di definire lo zoom da associare al controllo
- .NTSSetRichiesto(
 - evidenzia il campo
- InitControlsEndInit(Me, True)
 - riattiva gli eventi dei controlli UI

- E' obbligatoria per ogni GridView inserire la seguente:
grvCiva.NTSSetParam(...
- Per ogni colonna invece le NTSSetParam sono tipizzate quindi il suffisso della NTSSetParam scelta determina il tipo di colonna (testo, numero, ...)

```
tb_aliq.NTSSetParamNUM(oMenu, oApp.Tr(Me, 128230023373093533, "Aliquota"), oApp.FormatSconti, 6, 0, 100)
tb_codciva.NTSSetParamNUMTab(oMenu, oApp.Tr(Me, 128230023372625014, "Cod. IVA"), tabciva)
tb_desciva.NTSSetParamSTR(oMenu, oApp.Tr(Me, 128230023372781187, "Descr. IVA"), 20, False)
tb_tipacq.NTSSetParamSTRTab(oMenu, oApp.Tr(Me, 128230023374030571, "Tipo oper. IVA"), tabtacq, False)
tb_vp11.NTSSetParamCHK(oMenu, oApp.Tr(Me, 128230023374811436, "VP1/1"), "S", "N")
    .NTSSetParamDATA (
```

- I controlli invece sono già tipizzati TextBoxNum, TextBoxStr, TextBoxData, ... quindi per questi le NTSSetParam non hanno suffisso



```
BN_PAGA.VB
FRM_PAGA
InitControls

'-----
'completo le informazioni dei i controlli
edTb_codpaga.NTSSetParamTab (oMenu, oApp.Tr (Me, 128230023476167713, "Codice pagamento"), tabpaga)
edTb_despaga.NTSSetParam (oMenu, oApp.Tr (Me, 128230023476323886, "Descrizione pagamento"), 50, False)
edTb_datafdec.NTSSetParam (oMenu, oApp.Tr (Me, 128230023476480059, "Data fissa di scadenza"), True)
edTb_numrate.NTSSetParam (oMenu, oApp.Tr (Me, 128230023476636232, "Numero delle rate"), "0", 2, 0, 60)
edTb_gioiniz.NTSSetParam (oMenu, oApp.Tr (Me, 128230023476792405, "Prima rata"), "0", 3, 0, 999)
edTb_gioiniz2.NTSSetParam (oMenu, oApp.Tr (Me, 128230023476948578, "Seconda rata"), "0", 3, 0, 999)
edTb_gioperi.NTSSetParam (oMenu, oApp.Tr (Me, 128230023477104751, "Periodicità"), "0", 3, 0, 999)
edTb_scopaga.NTSSetParam (oMenu, oApp.Tr (Me, 128230023477260924, "Sconto pagamento"), "0.00", 5, -100, 100)
edTb_speinca.NTSSetParam (oMenu, oApp.Tr (Me, 128230023477417097, "Spese incasso"), oApp.FormatImporti, 13, 0, 999)
edTb_concassp.NTSSetParamTab (oMenu, oApp.Tr (Me, 128230023477573270, "Contropartita vendite/acquisti"), tabcove)
edTb_mesees1.NTSSetParam (oMenu, oApp.Tr (Me, 128230023477729443, "Primo mese escluso"), "0", 2, 0, 12)
edTb_mesees2.NTSSetParam (oMenu, oApp.Tr (Me, 128230023477885616, "Secondo mese escluso"), "0", 2, 0, 12)
edTb_giofiss.NTSSetParam (oMenu, oApp.Tr (Me, 128230023478041789, "Giorno fisso"), "0", 2, 0, 31)
edTb_codstpg.NTSSetParamTab (oMenu, oApp.Tr (Me, 129436398215341834, "Sottotipo pagamento"), tabstpg)
edTb_codmodi.NTSSetParamTab (oMenu, oApp.Tr (Me, 129453723171660157, "Modalità di incasso intrastat"), tabmodi, F

ckTb_shiftm.NTSSetParam (oMenu, oApp.Tr (Me, 128230023478197962, "Sposta rate successive"), "S", "N")
ckTb_flcondp.NTSSetParam (oMenu, oApp.Tr (Me, 128230023478354135, "Condizioni particolari"), "S", "N")

cbTb_tipincecr.DataSource = dttTipInc
cbTb_tipincecr.ValueMember = "cod"
cbTb_tipincecr.DisplayMember = "val"
cbTb_tipincecr.NTSSetParam (oApp.Tr (Me, 128230023478510308, "Tipo incasso"))
```

- Combo

```
Dim dttTipiva As New DataTable()
dttTipiva.Columns.Add("cod", GetType(Short))
dttTipiva.Columns.Add("val", GetType(String))
dttTipiva.Rows.Add(New Object() {1, "Op. imponibili"})
dttTipiva.Rows.Add(New Object() {2, "Op. esenti/non imponib."})
dttTipiva.Rows.Add(New Object() {3, "Acq. art. 8 2° comma"})
dttTipiva.Rows.Add(New Object() {4, "Fuori campo IVA"})
dttTipiva.Rows.Add(New Object() {6, "*****"})
dttTipiva.Rows.Add(New Object() {7, "*****"})
dttTipiva.Rows.Add(New Object() {8, "Da ventilare"})
dttTipiva.AcceptChanges()

Dim dttTipel As New DataTable()
dttTipel.Columns.Add("cod", GetType(String))
dttTipel.Columns.Add("val", GetType(String))
dttTipel.Rows.Add(New Object() {"A", "Acquisti"})
dttTipel.Rows.Add(New Object() {"V", "Vendite"})
dttTipel.AcceptChanges()

tb_tipiva.NTSSetParamCMB(oMenu, oApp.Tr(Me, 128230023372937360, "Tipo IVA"), dttTipiva, "val", "cod")
tb_tipel.NTSSetParamCMB(oMenu, oApp.Tr(Me, 128230023373562052, "Tipo elenco"), dttTipel, "val", "cod")
NTSSetParamCMB (ByRef Menu As NTSInformatica.CLE_MENU, NomeCampo As String, dttValue As System.Data.DataTable, strColDisplay As String,
strColValue As String)
```

- Check

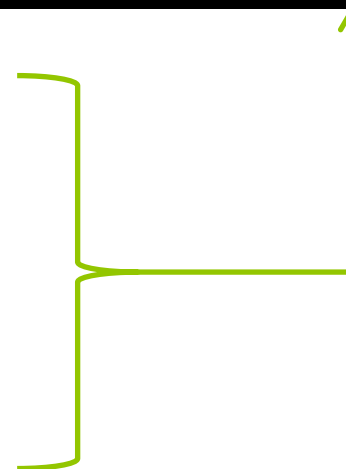
```
tb_volaff.NTSSetParamCHK(oMenu, oApp.Tr(Me, 128230023375436128, "Incr. volume affari"), "S", "N")
NTSSetParamCHK (ByRef Menu As NTSInformatica.CLE_MENU, NomeCampo As String, ValoreCheck As Object, ValoreUnCheck As Object)
```

- Date

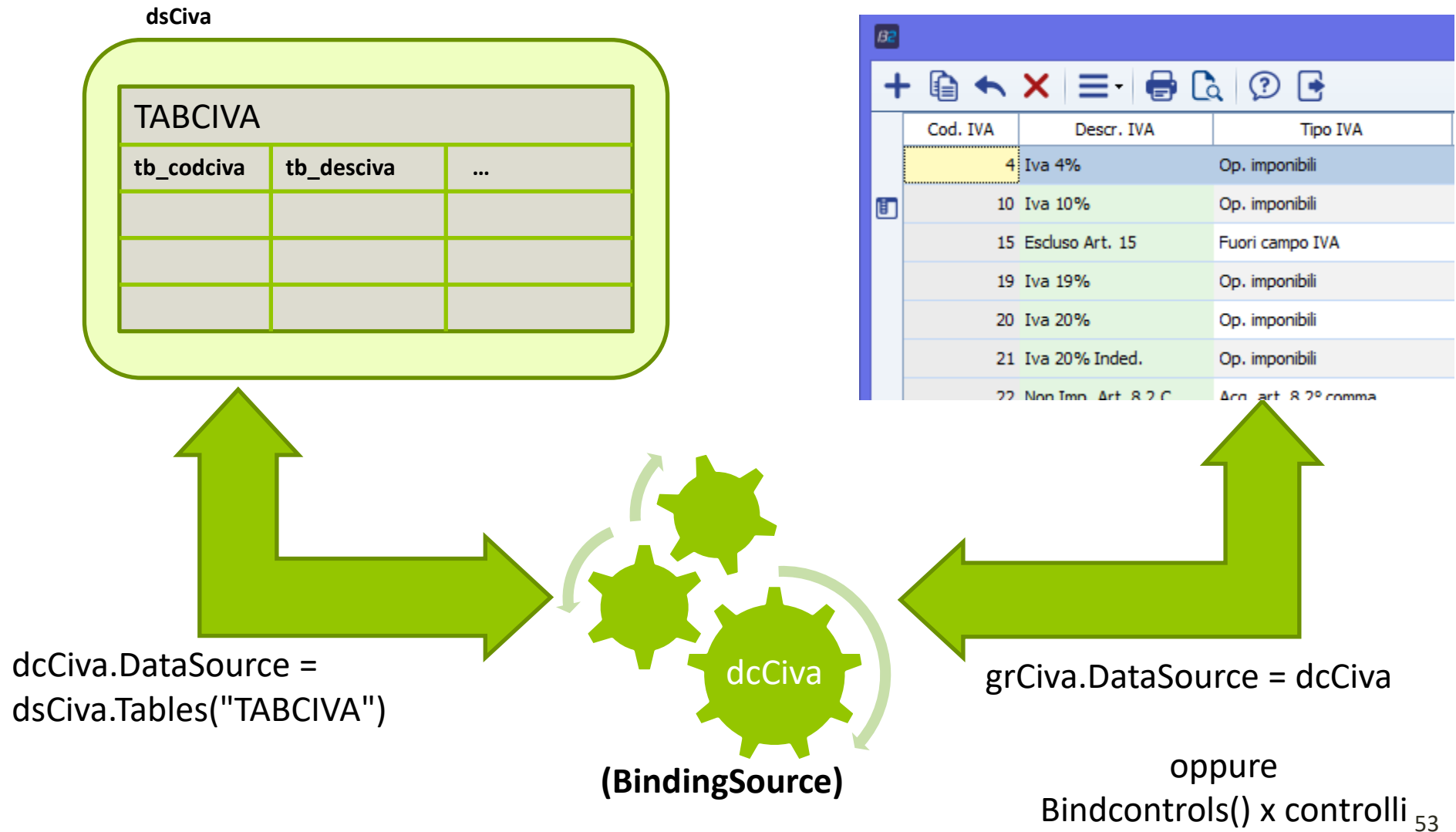
```
.NTSSetParamDATA(oMenu, oApp.Tr(Me, 128230023374811436, "per esempio"),  
▲ 1 di 2 ▼ NTSSetParamDATA (ByRef Menu As NTSInformatica.CLE_MENU, NomeCampo As String, AccettaNull As Boolean)  
▲ 2 di 2 ▼ NTSSetParamDATA (ByRef Menu As NTSInformatica.CLE_MENU, NomeCampo As String, AccettaNull As Boolean, Tipo As String, dtMin As Date,  
dtMax As Date)
```

- Tipo:

- "D" 'data (default)
- "O" 'ora + minuti + secondi
- "M" 'ore + minuti
- "X" 'data + ora + minuti + secondi



- Combinazione ctrl + shift + alt +f12
- Genera le primitive NTSSetParam da inserire nella initcontrols() e le istruzioni da inserire nella bindcontrols() dei controlli che ancorano non le hanno



- Nei controlli (Textbox, Checkbox, ... non griglia) l'associazione fra controlli e BindingSource avviene tramite la Bindcontrols, dove controllo per controllo viene indicato a quale *nome_datatable.nome_colonna* collegare il campo

```
Public Overridable Sub Bindcontrols()  
    Try  
        '-----  
        'se i controlli erano già stati precedentemente collegati, li scollego  
        NTSTFormClearDataBinding(Me)  
        '-----  
        'collego il BindingSource ai vari controlli  
        edTb_codpaga.NTSDbField = "tabpaga.tb_codpaga"  
        edTb_despaga.NTSDbField = "tabpaga.tb_despaga"  
        ...  
        ckTb_shiftm.NTSText.NTSDbField = "tabpaga.tb_shiftm"  
        ...  
        '-----  
        'per agganciare al dataset i vari controlli  
        NTSTFormAddDataBinding(dcPaga, Me)  
    End Try  
End Sub
```

- Nelle griglie i nomi delle varie colonne (NTSGridColumn) devono rispecchiare obbligatoriamente i nomi colonne del datatable
- Il nome di un controllo potrebbe essere uno qualsiasi, ma Nts segue la seguente convenzione prefisso del controllo (ed, ck, ...) + nome colonna del datatable

BeforeColUpdate e AfterColUpdate

```
Public Overrides Function Apri(
```

```
...
```

```
dsShared = ds
```

```
AddHandler dsShared.Tables(strNomeTabella).ColumnChanging, AddressOf BeforeColUpdate
```

```
AddHandler dsShared.Tables(strNomeTabella).ColumnChanged, AddressOf AfterColUpdate
```

```
bHasChanges = False
```

```
...
```

```
Public Overrides Sub BeforeColUpdate(
```

```
...
```

```
'cerco e, se la trovo, eseguo la funzione specifica per la colonna modificata
```

```
Dim strFunction As String = "BeforeColUpdate_" & e.Column.ColumnName.ToLower
```

```
Dim fun As System.Reflection.MethodInfo = Me.GetType.GetMethod(strFunction)
```

```
If Not fun Is Nothing Then fun.Invoke(Me, New Object() {sender, e})
```

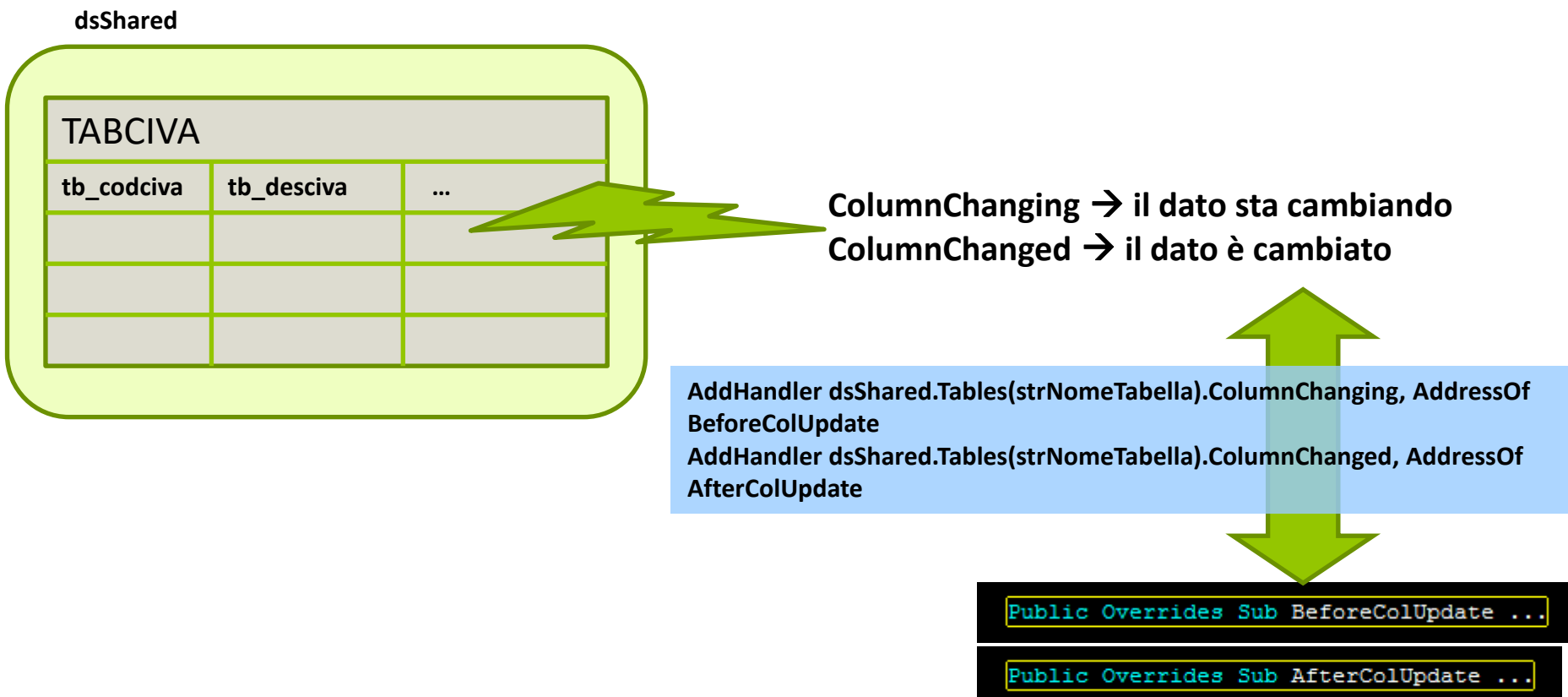
```
...
```

```
Public Overrides Sub AfterColUpdate(
```

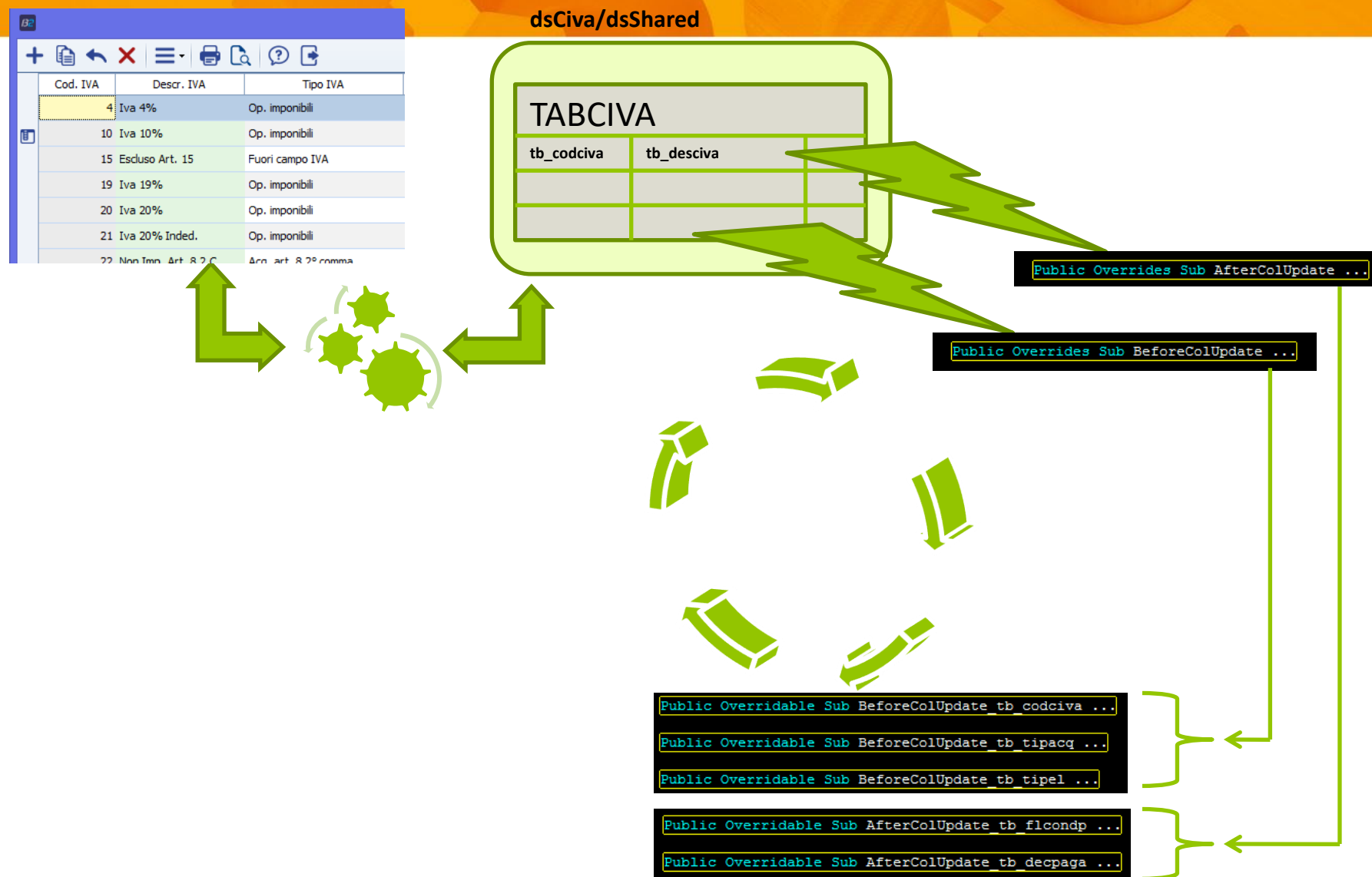
```
...
```

```
bHasChanges = True
```

```
...
```



Quindi l'ordine di esecuzione è sempre prima la BeforeColUpdate e poi eventualmente AfterColUpdate.



- In molte form semplici nella toolbar non c'è il pulsante salva, in questo caso è Business che salva in automatico senza chiedere niente al cambio di record o in uscita dalla form, il comando salva c'è solo su child complessi (veboll/orgsor/crgsof/cgprin/...) per consentire un salvataggio a discrezione dell'operatore.
- Sulla base di quanto detto è disponibile l'opzione di registro globale OPZIONI\SalvaChiediConf (default 0). L'opzione se impostata a 0 non chiede se salvare, se impostata a -1 chiede se salvare, se impostata a 2 chiede se salvare solo se non è un record di griglia.
- La stessa opzione è disponibile anche per singolo programma con gli stessi valori della precedente, per esempio BS--PAGA\OPZIONI\SalvaChiediConf, che serve per tornare al vecchio comportamento.
- Per gestire il caso di utenti non abilitati a salvare/cancellare da CTRL+SHIFT+Click SX sulla form è possibile impostare 'allow update' e 'allow delete' per programmi. Prima di salvare un record o cancellarlo il framework fa un controllo e, nel caso non l'operatore non sia abilitato, viene dato un messaggio di blocco.
- Usando la funzioni **NTSChiediCancella** e **NTSChiediSalva** viene standardizzato il modo in cui si chiede di salvare un record all'uscita dalla form o al cambio di record: al cambio di record la domanda dovrà essere sempre 'vuoi salvare si/no', default 'si', mentre in chiusura di form sarà 'vuoi salvare si/no/cancel' con default 'cancel'. E valutano le configurazioni fatte con il CTRL+SHIFT+Click SX
- NB: form che gestiscono una tabella non in modalità griglia (per esempio BN__PAGA) quando si preme 'nuovo' il bHasCHnages va settato a False, altrimenti se la tabella è vuota, come entro la prima volta viene proposto un nuovo record ma se non si vuole salvare non è possibile uscire.

- Le form possono essere al massimo 700 x 550 in design controllato da framework (viene ignorato solo se la form è maggiore o uguale a 1400x1000).
- In realtà non è che non si può allargare per esempio un programma a 800 x 600, il limite è in fase di design, nel senso che se per esempio abbiamo una maschera che lavora bene 800 x 600, si può tranquillamente impostare per esempio a 1800 x 1600 in modo che in progettazione sia facilmente disegnabile, per poi portarla a 700 x 550. Non viene tagliato nulla visto che ora su cube i panel sono attivabili come scrollabili.
- Il vincolo di 700 x 550 viene ignorato se la form ha una width (larghezza) che finisce per 1, provare ad impostare in width anzichè 801*550. Non c'è invece questa possibilità di regola dell'uno in height (altezza).
- Poi ovviamente l'utente finale utilizzatore, dopo il primo avvio che magari è a 700 x 550 preme per portarla a schermo intero, o magari la ridimensiona a suo piacere a 3/4 di schermo lavorando sui bordi. Dai successivi avvii l'utente trova la maschera come l'aveva lasciata.
- Se proprio al primo avvio non piace averla a 700 x 550, si può cablare da codice nella activate first di metterla a 800 x 600 se questa è inferiore a 800 x 600.

- la distanza dal bordo sinistro dal parent è 4 pixel (gestito in automatico)
- la distanza dal bordo superiore del parent è 4 pixel (gestito in automatico); se il font del controllo è diverso da quello standard (8) la non ci sono vincoli di TOP
- la distanza dal bordo inferiore del parent è 4 pixel **(gestito a mano)**
- la distanza dal bordo destro dal parent è 4 pixel **(gestito a mano)**
- la distanza dal controllo successivo deve essere 4 pixel
- i controlli per default sono alti 20.
- gli NTSTButton 26 (possono essere 20 solo se Button.Text = "...")
- l'altezza dei controlli da 20 può passare a multipli di 24 (20, 48, 72, ...); solo per NTSPanel e NTSTGroupbox no Tile l'altezza è multipla di 4. Se il font del controllo è diverso da quello standard (8) la non ci sono vincoli di HEIGHT
- l'altezza dei NTSTButton è 26 o 40 o 44 o 60 o 100; oltre è libera
- la larghezza dei controlli è a multiplo di 4
- per i controlli XX e per gli NTSPanel dove il posizionamento deve essere più mirato (vedi BNXXNODO, BNFWNODO, ...) per non far scatenare gli automatismi di cui sopra basta impostare sull'NTSUserControl/NTSPanel la proprietà FreeLayout = True
- ovviamente alcune note di cui sopra non vengono attivate nel caso di controlli con Dock impostato

Business cube deve poter lavorare con caratteri grandi, ci sono però alcune regole da seguire per far funzionare bene il tutto con caratteri grandi:

- è sbagliato far dipendere le dimensioni di controlli/form da un'immagine (per esempio, `form.size = immagine.size`) o da altro
- le immagini vanno caricare in modo 'intelligente', vedere slide successive
- **massimizzare l'utilizzo di font standard, senza grassetto, sottolineato, barrato, dimensioni più grandi, ...** in diversi casi la rappresentazione grafica è pessima con dpi 125, 150, ... questo è dovuto al fatto che con dpi 96 il font di default è Tahoma, mentre passando a 120 dpi o superiore, il font viene convertito in Microsoft San Serif (che è il font di default di Windows)
- la routine standard di dimensione/posizione controlli non lavora se si ha una form con nome che non inizia per FRM (attenzione a form personalizzate, Business non esistono)

- Nella toolbar, a design, mettere il comando ZOOM come sempre (tlbZoom), ma il framework nella form_load lo sparire (ci pensa la tlbMain.NTSSetToolTip()), se va mantenuto (per esempio BNCRGLEA per le griglie) sarà alla fine della form_load che dovrà essere reso nuovamente visibile da codice.
- In molte form con solo una griglia CODICE/DESCRIZIONE sono stati tolti se presenti i comandi stampa, stampa video e imposta stampante (si può stampare tranquillamente la griglia senza report).
- Presenza del flow layout control su tutti molti programmi eccetto che sulle griglie.
- Presenza della barra di navigazione a sinistra e delle wings a destra.
- Cercato di limitare al massimo le form modali e alcune c'è la possibilità inserirle come risultato in un tab dei risultati.
- Tolte le form standard _T e _F, ma il meccanismo è ancora presente (personalizzazioni).

Su devexpress 6.2.2.2 il colore trasparente delle immagini era il pixel in basso a SX. Ora non lo è più, per cui tutte le immagini sono state riprese per togliere il rosso che serviva per impostare il colore trasparente, se avete immagini vanno rifatte senza sfondo rosso.

Per caricare immagini in base ai font grandi di windows:

- creare le immagini con le dimensioni 20x20, 24x24, 32x32, 48x48 e metterle nella BNIMAGES
- inoltre deve venir predisposta anche l'icona 96x96, non utilizzata (per ora) da bus, ma potrà essere usata per essere convertita in immagine vettoriale con cui poi si potranno generare tutte le altre dimensioni mancanti in formato PNG.
- per tutte le immagini presenti in Business utilizzare come sistema per caricarle .ImagePath, .GlyphPath o .BackgroundImagePath, sarà il framework, al momento in cui deve venir caricata l'immagine, a scegliere in base ai DPI se usare la 24, 32, ... solo per i tipi Image occorre caricare le immagini tramite **'NTSBitmapFromFile'**
- per le icone dei treeview, oltre a creare le immagini e metterle nella BNIMAGES, bisogna

```
;' Crea la lista delle immagini
oApp.NTSSetImageListSize("TREEVIEW", ImageList1)
ImageList1.Images.Add("", oApp.NTSBitmapFromFile("TREEVIEW", oApp.ChildImageDir & "\\tr_closed.png"))
ImageList1.Images.Add("", oApp.NTSBitmapFromFile("TREEVIEW", oApp.ChildImageDir & "\\tr_open.png"))
trClas.ImageList = ImageList1
trClas.ImageIndex = 0
trClas.SelectedImageIndex = 1
```

NTSPictureBox

- Public Property ImagePath() As String
- Public Property BackgroundImagePath() As String

NTSLabel

- Public Property ImagePath() As String
- Public Property BackgroundImagePath() As String

NTSButton

- Public Property ImagePath() As String

NTSTabPage

- Public Property ImagePath() As String

NTSBarItem

NTSBarItem

NTSBarItem

- Public Property GlyphPath() As String

Invece la funzione ad esempio `oApp.NTSBitmapFromFile("TREEVIEW", oApp.ChildImageDir & "\tr_open.png")` come parametro ha una stringa che va indicata solo nel caso in cui si lavori con immagini per `NTSTreeView`, negli altri casi si può indicare sempre `""`, per esempio: `oImage = oApp.NTSBitmapFromFile("", oApp.ChildImageDir & "\info.png")`

- Per le regole che ha Business CUBE (DPI ADWARE) le label devono essere non autosize e lasciando un po' di spazio in più rispetto al testo (utile anche per internazionale), di default Height 20 pixel
- Non viene applicato il default se la Height del controllo è maggiore di 25 (testo che va a capo), lo stesso vale per CheckBox, Radiobutton e Button, in questi casi però non si guarda se la Height del controllo è maggiore di 25, ma se la proprietà `Appearance.TextOptions.WordWrap=DevExpress.Utils.WordWrap.Wrap`
- Label con Width o Height minore di 4 pixel sono libere (label usate come linee di separazione orizzontale o verticale)
- Se la NTSLabel è alta fino a 30 il testo può essere allineato solo middleleft, midlecenter e midleright
- Aggiunte nuove proprietà NTSEdgeStyle (valori FieldCaption, FieldDescription)

NTSButton con sola immagine la proprietà 'ImageToTextAlignment' deve essere settata a DevExpress.XtraEditors.ImageAlignToText.LeftCenter' altrimenti l'immagine non è al centro

Aggiunta negli NTSCheckBox la proprietà NTSCheckedValue che ritorna il valore associato allo stato del CheckBox. Ritournerà i valori impostati con la NTSSetParam, per esempio:

```
!xx_valore = ckValore.NTSCheckedValue
```

Invece di usare:

```
!xx_valore = If(ckValore.Checked, "S", "N")
```

Ora gli NTSPanel sono scrollabili, nuove voci di NTSAutoScroll, NTSVerticalScroll e NTSHorizontalScroll (per un esempio vedere BNXXFLDO)

Se l'immagine è più grande della NTSPictureBox, quando ci si clicca sopra si ingrandisce, per mantenere il vecchio comportamento (immagine statica) settare a false:

- `Properties.AllowScrollOnMouseWheel=DevExpress.Utils.DefaultBoolean.False`
- `Properties.AllowScrollViaMouseDrag=DevExpress.Utils.DefaultBoolean.False`
- `Properties.AllowZoomOnMouseWheel=DevExpress.Utils.DefaultBoolean.False`

Su NTSListbox non impostare 'selectionmode = none' in quanto non fa selezionare nessun voce e se ci si clicca sopra non fa scatenare eventi

- I Tile sono i NTSTGroupBox non c'è un nuovo tipo di controllo, si chiamano così quando sono dentro ad un NTSTFlowLayoutPanel, occorre solo settare la proprietà Tile=True (se si restringe troppo il tile «perde» la caption si agisce sulla proprietà ShowCaption = True/False)
- non è possibile inserire NTSTGroupBox dentro altri NTSTGroupBox

- Va configurato tramite NTSTSetParam come si fa per la NTSTGridView
- `flPaga.NTSTSetParam(oMenu, oApp.Tr(Me, 130916282270586823, ""))`

Aggiunte costanti per identificare una:

- nuova riga (-999998)
- nessuna riga (-999999)

nota: si possono comunque usare le vecchie istruzioni!

Quindi anzichè scrivere:

```
If grvRef.FocusedRowHandle = -999998 Then
```

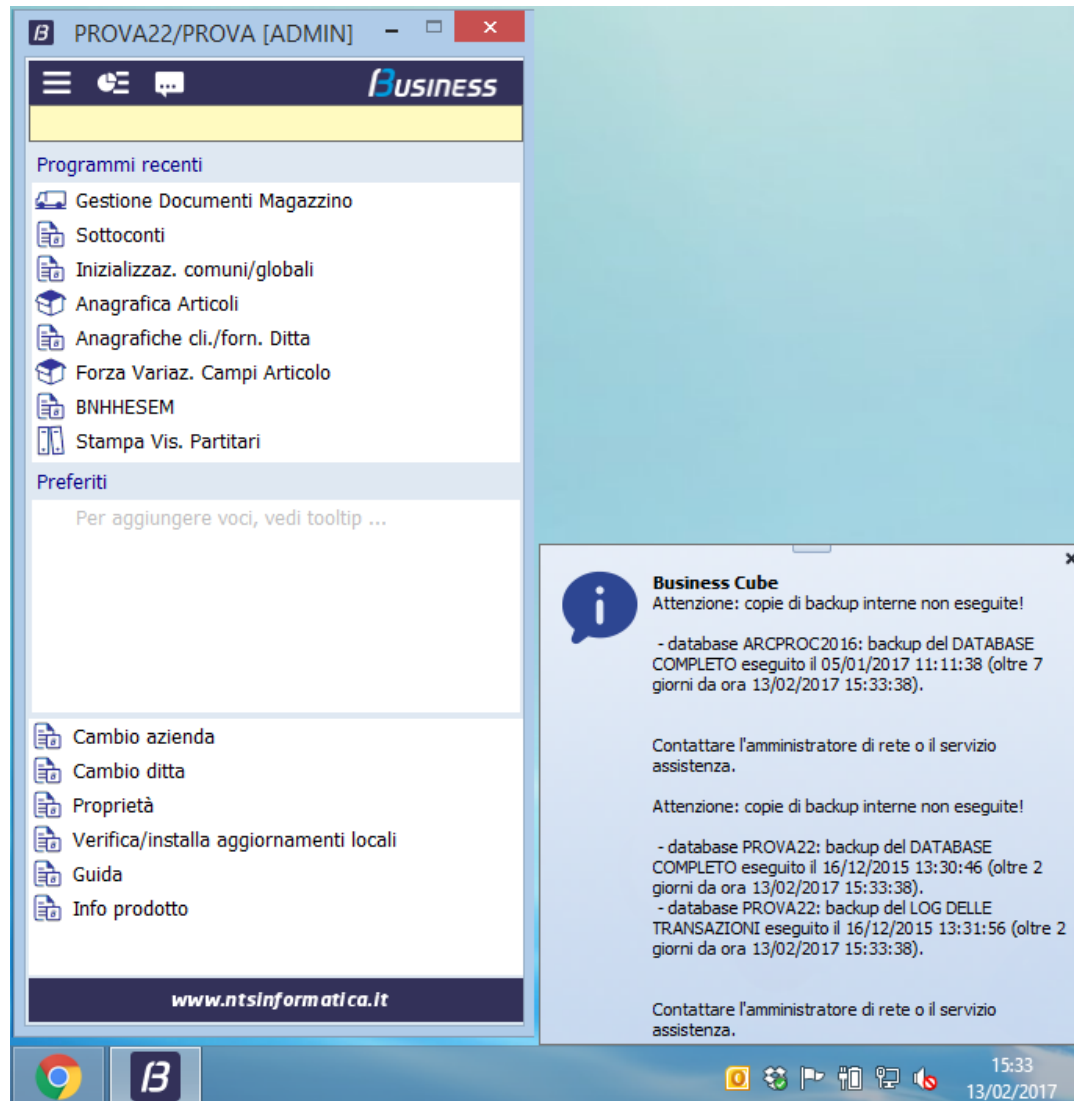
```
If grvRef.FocusedRowHandle = -999999 Then
```

Si può scrivere:

```
If grvRef.FocusedRowHandle = NTSGridView.NUOVA_RIGA Then
```

```
If grvRef.FocusedRowHandle = NTSGridView.NESSUNA_RIGA Then
```

Tolta l'icona sulla taskbar di windows



- Tolta l'icona sulla taskbar di windows per menu/alert e sostituita con nuovo oggetto NTSAAlertPopup di FRM__CHIL.
- Possiamo usare da codice per fare apparire un messaggio tramite l'istruzione:
 - NTSShowPopUp("Attenzione: copie di backup interne non eseguite!")

Business CUBE, nuovo metodo ExecuteMassiveInsert

Con Business CUBE la versione minima di SQL che supportiamo è la 2008, da questa versione è possibile eseguire query di **INSERT INTO** con più **VALUES**, per esempio:

```
INSERT INTO tabpaga (tb_codpaga, tb_despaga) VALUES (1, 'R.B.'), (2, 'Bancomat'), (3, 'Contanti'), ...
```

Il massimo numero di record che si possono inserire con una singola query da SQL è 1000, però a seguito di alcune ricerche e di alcune prove si è visto che nonostante sia possibile inserire fino a 1000 record per query, le prestazioni migliori si ottengono inserendo fino a 50 record alla volta (poi iniziano a calare, nonostante rimangano superiori all'inserimento di un record alla volta).

Per questo motivo è stato creato un nuovo metodo **ExecuteMassiveInsert**, presente nel CId__base e quindi disponibile in tutti i BD di Business, che passandogli un DataTable procede all'inserimento di tutte le righe in esso contenute.

In pratica sarà possibile sostituire il codice:

```
strSQL = "INSERT INTO prinet " & GetQueryInsertField(dttPn, "pn_")  
For i = 0 To dttPn.Rows.Count - 1  
    strSQLVal = GetQueryInsertValue(dttPn, dttPn.Rows(i), "pn_")  
    Execute(strSQL & " VALUES " & strSQLVal, CLE__APP.DBTIPO.DBAZI, dbConn)  
Next
```

Con:

```
ExecuteMassiveInsert("prinet", dttPn, "pn_", CLE__APP.DBTIPO.DBAZI, dbConn)
```

ATTENZIONE: in caso procediate alla sostituzione di codice esistente, il metodo fa la insert di **TUTTE** le righe del **DataTable**, non viene effettuato alcun filtro sullo stato delle righe del DataTable.

Quando si cambia il datatable al bindingsource che successivamente dovrà essere agganciato al datasource della griglia, prima occorre scollegare il bindingsource dalla griglia stessa, diversamente possono verificarsi degli errori tipo:

'La colonna 'dd_riga' non appartiene alla tabella CICLI.
(error type: ArgumentException)'

Ad esempio in BNDBDIBA la funzione apri, legge il datatable dsCicli, lo collega al bindingsource dcCicli e successivamente il dcCicli viene collegato alla griglia grCicli. Alla prima apertura di distinta base tutto opera correttamente, visto che quando viene settato il dcCicli quest'ultimo non è ancora collegato alla griglia. A questo punto mi posiziono nella riga 4 della griglia dei cicli e senza uscire dal programma, utilizzo ancora la funzione apri per aprire un'altra DB che ha solo 1 lavorazione. Nel momento in cui si esegue il comando:

```
dcCicli.DataSource = dsCicli.Tables("CICLI")
```

se precedentemente non è stato settato il comando **grCicli.DataSource = Nothing** viene scatenato un errore.

La sintassi corretta è:

```
grCicli.DataSource = Nothing
```

```
dcCicli.DataSource = dsCicli.Tables("CICLI")
```

```
grCicli.DataSource = dcCicli
```


oCldCiva.LeggiTabCiva(dsShared)
AddHandler dsShared.Tables("TABCIVA").ColumnChanging, AddressOf BeforeColUpdate
AddHandler dsShared.Tables("TABCIVA").ColumnChanged, AddressOf AfterColUpdate

SI

```
Public Overridable Function LeggiTabCiva(ByRef dsOut As DataSet) As Boolean
    Dim strSQL As String = ""
    strSQL = " SELECT tabciva FROM tabciva"
    'Chiedo i dati al database
    dsOut = OpenRecordset(strSQL, CLE__APP.DBTIPO.DBAZI, "TABCIVA")
```

NO

```
Public Overridable Function LeggiTabCiva(ByRef dsOut As DataSet) As Boolean
    Dim strSQL As String = ""
    dsOut.Tables("TABCIVA").Rows.Clear()
    strSQL = " SELECT tabciva FROM tabciva"
    'Chiedo i dati al database
    For Each dtrRow As DataRow In OpenRecordset(strSQL, CLE__APP.DBTIPO.DBAZI, "TABCIVA").Tables("TABCIVA").Rows
        dsOut.Tables("TABCIVA").ImportRow(dtrRow)
    Next
```

1° soluzione

```
dsOut.Tables.Remove("TABCIVA")
dsOut.Tables.Add("TABCIVA")
```

2° soluzione

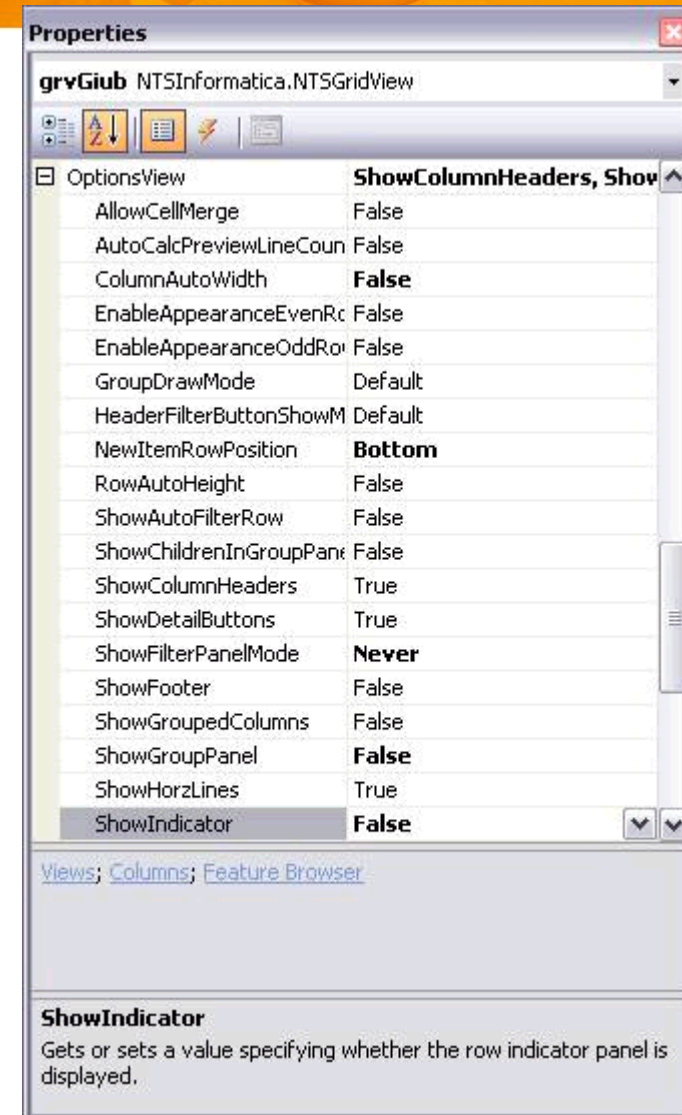
```
RemoveHandler dsShared.Tables("TABCIVA").ColumnChanging, AddressOf BeforeColUpdate
RemoveHandler dsShared.Tables("TABCIVA").ColumnChanged, AddressOf AfterColUpdate
```

BN__STD.CLN__STD:

- FRM__CHIL
- CLE__MENU
- CLE__FRWK
- Librerie di menu (CLELBMENU)
- Funzioni di entity (CLE__BASE\CLE__BASN)
- Funzioni DAL (CLD__BASE\CLDLBBASE)
- Le stampe (CLE__CRPE)
- Gli zoom (CLE__CZOO)

- Controlli devexpress rimappati (superclassing dei controlli devexpress)
- Form di base di partenza da ereditare per lo sviluppo dell'interfaccia UI di programmi
- DittaCorrente, oApp, oMenu, ...
- NTSCallStandardZoom
- NTSCFindControlByName
- oScript , NTSScriptExec
- GCTL....

- È possibile nascondere il menù con le funzioni di griglia (per intenderci quello sempre visibile a sinistra di ogni griglia) per i programmi nei quali questa funzionalità non è necessaria e allo stesso tempo c'è esigenza di guadagnare spazio (ad esempio per i programmi di Picking su palmare).
- Per nascondere la barra a sinistra è necessario settare a false "ShowIndicator" presente tra le "OptionsView".



Aggiunte nuove proprietà ai TextBox:

- TextBoxNum -> .TextInt e .TextDec
- TextBoxData -> .TextData
- TextBoxStr -> .TextStr

Queste nuove proprietà consentono di passare i valori ai TextBox senza dover fare il cast al tipo stringa o di ritornare il valore di un textbox dal tipo stringa al tipo corretto. L'utilizzo di queste proprietà va a trasferire i dati direttamente al datatable collegato (se presente), in maniera simile a come viene fatto dalla NTSTextDB. Esempio:

Prima:

- edTb_anno.NTSTextDB = NTSCStr(Now.Year)
- dim lAnno as integer = NTSCInt(edTb_anno.text)

Adesso:

- edTb_anno.TextInt = Now.Year
- dim lAnno as integer = edTb_anno.TextInt

Per campi textbox e colonne di griglia con SmartArt ora è possibile definire se un textbox o una colonna di griglia gestisce gli smartart, con le nuove proprietà:

NTSSmartArt

NTSSmartArt_New

NTSSmartArt_Open

NTSSmartArt_Zoom

NTSSmartArt_Recent

NTSSmartArt_Menu

Implementazione nuove oApp.MsgBoxExclamation".

- GetSettingBus, SaveSettingBus
- RunChild
- ValCodiceDB
- ReportPEInit (rimappa in sostanza le funzioni della CLE__CRPE)
- oApp
- oCleComm

```
Dim oPar As New CLE__CLDP  
oPar.strParam = strParam  
oMenu.RunChild("NTSInformatica", "FRMORGSOR", "", DittaCorrente, "", "Frmorgsor",  
oPar, "", True, True)
```

- In pratica funziona come per la NtsInstanziaDLL

```
Dim frmSrpt As FRM__SRPT = Nothing  
Dim oPar As CLE__CLDP  
oPar = New CLE__CLDP  
frmSrpt = CType(NTSNewFormModal("FRM__SRPT"), FRM__SRPT)  
frmSrpt.Init(oMenu, oPar, DittaCorrente)  
frmSrpt.ShowDialog()
```

- Variabili di ambiente(CLE__APP - oApp)
 - oApp
- Variabili di logon
 - oApp.user
- Variabili di connessione
 - oApp.DB, oApp.DBAP
- Messaggistica
 - oApp.Msgbox, oApp.Inputbox, oApp.Tr, ...

NB: le InputBox accettano come strTipo anche la lettera “D” per validare come data il testo scritto dall’utente.
- Parametri child (CLE_CLDP - oCallParams)

- Solo dichiarazioni Static/Shared
- Non viene istanziato
- Viene sempre referenziato
- Gestione errori (GestError)
- CStrSQL, CDataSQL
- NTSCDec, NTSCInt, NTSCStr
- Costanti tabelle standard tabciva, tabcfam, ... (usate dalle NTSSetParam)
- IntSetDate, IntSetNum
- NtsIstanziaDLL
- GetSettingReg, SetSettingReg
- Costanti dei moduli: CLN__STD.bsModAll, ...

UI → oMenu.oCleComm

Entity → CType(oCleComm, CLELBMENU)

- Leggi\ScriviPrezzo\Sconti\Provv
- TrovaCodartDaCodart, TrovaCodartDaCodarfo, TrovaCodartDaBarcode
- Generazione files ABI/RID/MAV/BONIFICI
- ConvQuantUM, ...
- GeneraNumCommeca
- AggiornaPrezzi
- AggiornaListini

- Apri, Salva, Nuovo, Ripristina, SetDefaultValue, RecordsChanged, BeforeColUpdate, AfterColUpdate, TestPreSalva
- \
- File di log
 - ThrowRemoteEvent
 - Init (con istanziall per dal)

Funzioni di DAL (Cld__base\BDLBBASE)

- GetSettingBus, SaveSettingBus
- LegNuma, AggNuma
- ScriviTabellaSemplice, LeggiTabellaSemplice
- GetTblInstId, ResetTblInstId
- ValCodiceDb
- Execute
- OpenRecordset

- Metodi e proprietà della BE__CRPE (rimappate da BE__MENU):
 - ReportPEInit
 - PESetSelectionFormula
 - ReportPEVai

 - PESetFormula

Lanciare una stampa da un child (BN__PAGA)

```
Public Overridable Sub Stampa(ByVal nDestin As Integer)
    Dim nPjob As Object
    Dim nRis As Integer = 0
    Dim strCrpe As String = ""
    Dim i As Integer

    Try
        '-----
        'preparo il motore di stampa
        strCrpe = ""
        nPjob = oMenu.ReportPEInit(oApp.Ditta, Me, "BS--PAGA", "Reports1", " ", 0, nDestin, "BS--PAGA.RPT", False, "Forme di pagamento", False)
        If nPjob Is Nothing Then Return

        '-----
        'lancio tutti gli eventuali reports (gestisce già il multireport)
        For i = 1 To UBound(CType(nPjob, Array), 2)
            nRis = oMenu.PESetSelectionFormula(NTSCInt(CType(nPjob, Array).GetValue(0, i)), CrpeResolveFormula(Me, CStr(CType(nPjob, Array).GetValue(2, i)), strCrpe))
            nRis = oMenu.ReportPEVai(NTSCInt(CType(nPjob, Array).GetValue(0, i)))
        Next

    Catch ex As Exception
        '-----
        Dim strErr As String = CLN__STD.GestError(ex, Me, "", oApp.InfoError, oApp.ErrorLogFile, True)
        '-----
    End Try
End Sub
```

- Il nome di uno ZOOM nella maggior parte dei casi corrisponde a ZOOM più il nome della tabella da ricercare. Gli identificatori di zoom particolari si trovano nel file di framework BE__CZOO
- La comunicazione tra zoom e child avviene solitamente tramite la Ntscallstandardzoom() definita in FRM__CHIL, però è possibile anche chiamare lo zoom in modo custom tramite l'istruzione NTSZOOM.ZoomStrIn(...)
- Gli zoom hanno una Init con firma differente da quella di un programma child:

```
Public Overloads Function Init(ByRef Menu As CLE__MENU, ByVal NomeZoom As String,  
    ByRef Param As CLE__CLDP, Optional ByVal Ditta As String = "",  
    Optional ByRef SharedControls As CLE__EVNT = Nothing) As Boolean
```

Gli zoom (esempio di ri-mappatura alla chiamata standard di ricerca)

```
Public Overridable Sub tlbZoom_ItemClick(ByVal sender As System.Object, _
                                         ByVal e As DevExpress.XtraBars.ItemClickEventArgs) Handles tlbZoom.ItemClick
    Dim oParam As New CLE__CLDP

    Try
        If grvCiva.FocusedColumn.Name = "tb_codtpel" Then
            '-----
            'zoom tabsmel
            SetFastZoom(NTSCStr(grvCiva.EditingValue), oParam) 'abilito la gestione dello zoom veloce
            oParam.bVisGriglia = True
            oParam.strTipo = NTSCStr(grvCiva.GetFocusedRowCellValue("tb_tipel"))
            NTSZOOM.strIn = NTSCStr(grvCiva.EditingValue)
            NTSZOOM.ZoomStrIn("ZOOMTABSMEL", DittaCorrente, oParam)
            If NTSZOOM.strIn <> NTSCStr(grvCiva.EditingValue) Then grvCiva.SetFocusedValue(NTSZOOM.strIn)

        Else
            NTSCallStandardZoom()
        End If
    End Try
```

Gli zoom (esempio di ri-mappatura alla chiamata standard di gestione)

```
Public Overridable Sub tb_codtpel_NTSZoomGest(ByVal sender As System.Object, _
                                         ByRef e As NTSEventArgs) Handles tb_codtpel.NTSZoomGest

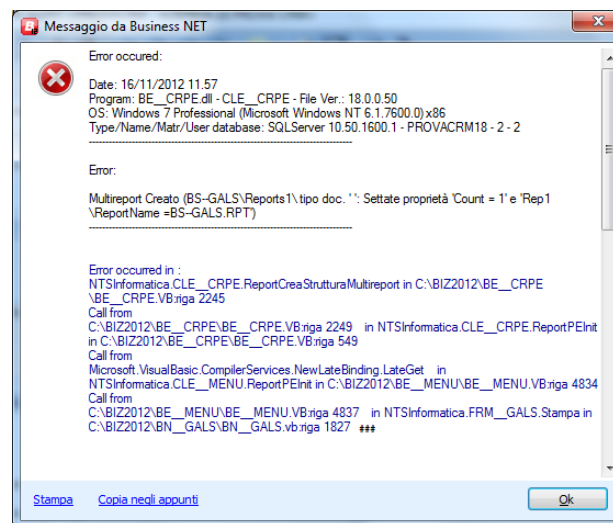
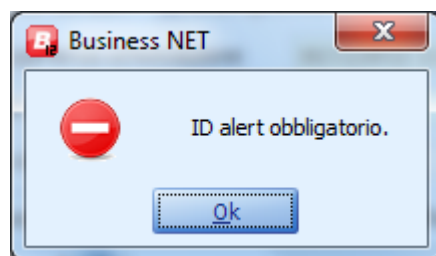
    Dim oCZoo As New CLE__CZOO
    Dim bNuovo As Boolean = True
    Dim oTmp As New Control
    Dim oPar As New CLE__CLDP
    Try
        Me.ValidaLastControl()

        If e.TipoEvento = "OPEN" Then
            bNuovo = False
            oPar.strParam = "APRI;"
        Else
            oPar.strParam = "NUOV;"
        End If
        oPar.strParam += NTSCStr(grvCiva.NTSGetCurrentDataRow!tb_tipel) & ";"
        oTmp.Text = NTSCStr(grvCiva.NTSGetCurrentDataRow!tb_codtpel)
        oPar.strParam += oTmp.Text

        NTSZOOM.OpenChildGest(oTmp, "ZOOMTABSMEL", DittaCorrente, bNuovo, oPar)

        e.ZoomHandled = True      'per non far lanciare la NTSZoomGest standard del controllo
        grCiva.Focus()
    Catch ex As Exception
        '-----
        Dim strErr As String = CLN__STD.GestError(ex, Me, "", oApp.InfoError, oApp.ErrorLogFile, True)
        '-----
    End Try
End Sub
```

- Gli standard di Try-Catch nei 3 diversi Layers
- La funzione ThrowRemoteEvent (sincrono, se qualcuno, nell'UI ascolta)
- Errori "classici" ed errori "applicativi"



Try

Catch ex As Exception

```
'-----  
CLN__STD.GestErr(ex, Me, "")  
'-----
```

End Try

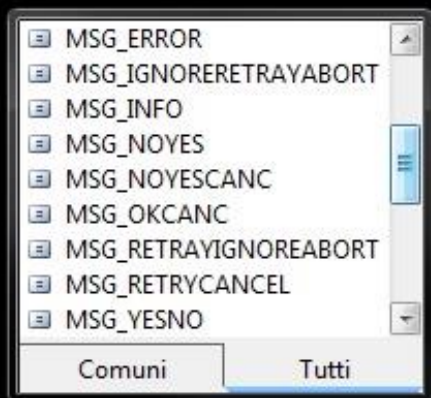
La funzione ThrowRemoteEvent (sincrono, se qualcuno, nell'UI ascolta)

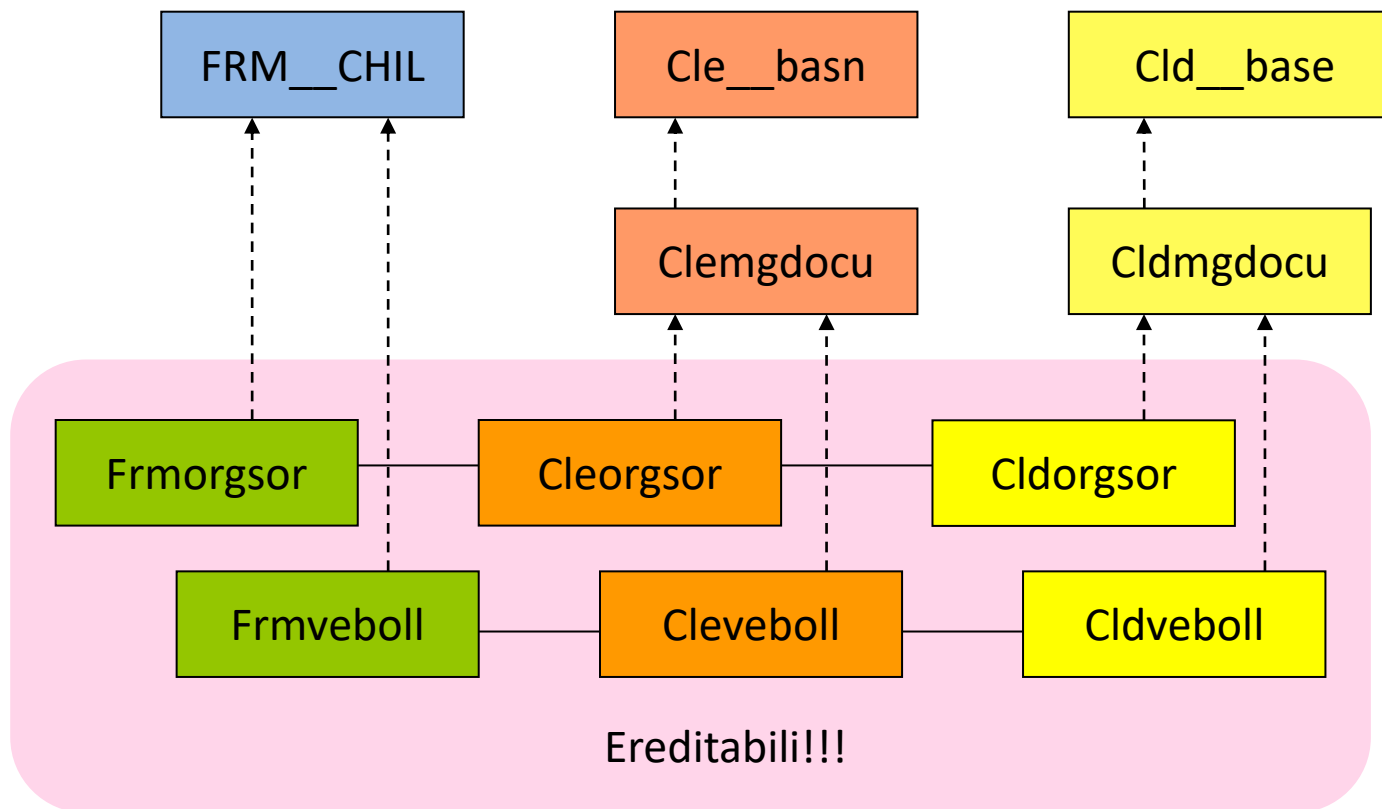
```
'questa da un messaggio con il divieto
ThrowRemoteEvent(New NTSEventArgs("", oApp.Tr(Me, 128344979226586147, "messaggio")))

'questa da un messaggio con l'info
ThrowRemoteEvent(New NTSEventArgs(CLN__STD.ThMsg.MSG_INFO, oApp.Tr(Me, 128344979226586147, "messaggio")))

'esempio per ritornare un valore dall'interfaccia
Dim evt As NTSEventArgs = Nothing
evt = New NTSEventArgs(CLN__STD.ThMsg.MSG_NOYES, oApp.Tr(Me, 128344979226586147, "messaggio"))
ThrowRemoteEvent(evt)
If Not evt.RetValue = CLN__STD.ThMsg.RETVALUE_YES Then
    Return False
End If

'esisto altri tipi selezionabili disponibili dalla classe ThMsg
CLN__STD.ThMsg.
```





— = istanza

↗ = eredita da

- astrazione documenti (ordini, magazzino, ecc)
- dttET (“TESTA”) (et_.....)
- dttEC (“CORPO”) (ec_.....)
- dttECIMP (NOME TABELLA DEL DB)

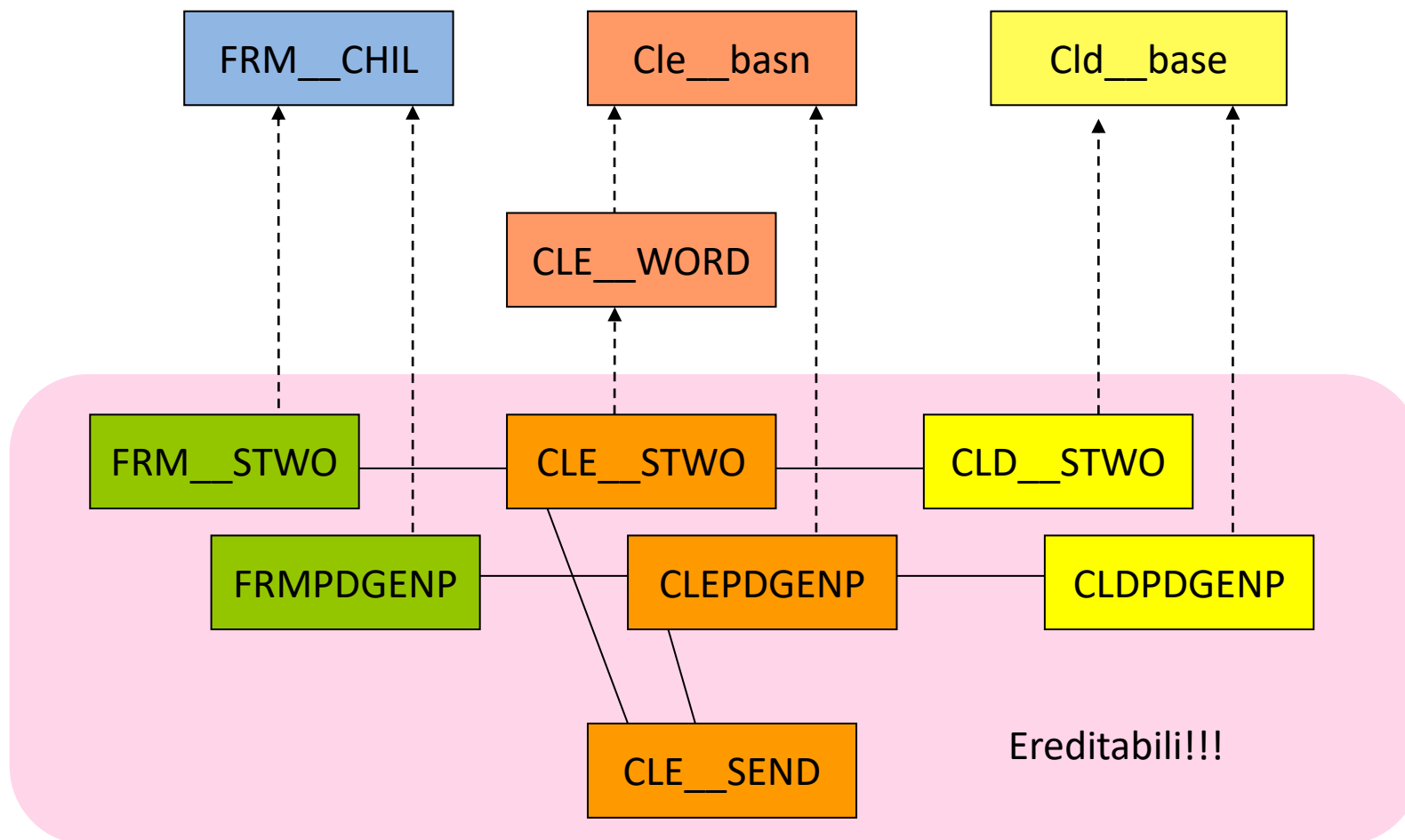
NB: tenere allineate le tabelle:

- testmag/testprb/testord
- movmag/movprb/movord
- movord/motrans (x la generazione di note di prelievo modificando la quantità assegnata della riga dell'ordine va a copiare la riga dell'ordine in motrans, con tutti i suoi campi)

E' possibile aggiungere controlli estesi su Frmveboll collegandoli alla tabella testmagc

- Aggiunto possibilità di esporre cono 'configurazione user interface' dei controlli aggiunti su Frmveboll collegati a testmagc
- I controlli devo essere collegati al datatable TESTAC e prefisso TM (es: nuovo NTSTextboxNum collegato al datatable TESTAC.tm_hhprova1, dove tm_hhprova1 è un nuovo campo numerico personalizzato aggiunto in testmagc)
- Per la validazione dei controlli sono disponibili le BeforeColUpdate_TESTAC_xxx e AfterColUpdate_TESTAC_xxx
- Da entity, per collegarsi alla riga della tabella testmagc del documento corrente è possibile utilizzare la variabile di tipo datatable DTTETC.Rows(0)!tm_tipork, simile a come si fa per la riga della tabella testmag del documento corrente DTTET.Rows(0)!et_tipork
-
- NB: per collegare i controlli a testmag la definizione è TESTA.ET_TIPORK, mentre per testmagc è TESTA.TM_TIPORK

Istanze e proprietari istanze



— = istanza

↖ = eredita da

Componenti del framework

Motore invio fax/email

- CLE__SEND

Motore stampa su word

- FRM__STWO, CLE__STWO, CLD__STWO (CLE__WORD)

Motore di stampa su PDF

FRMPDGENP, CLEPDGENP, CLDPDGENP

- Master-detail : BNCGUSAT
- Elaborativi : BNCGGNEF
- Di pura stampa : BNCGSTPN
- Gestione documento : BNORGSOR

- Realizzare un mini-zoom
- Realizzare un programma elaborativo/stampa

Scripting

Sviluppo di componenti verticali\personalizzati tramite ereditarietà

Editing

Configurazione Form & Griglie

Personalizzazione del DB

Strumento	Come funzionano	Come realizzarle
Scripting	.NTS	Editor di testo/Visual Studio
Sviluppo di componenti verticali personalizzati tramite ereditarietà	BO*.DLL\PDB DLLMAP.INI	Visual Studio
Editing	UICONF	Da business
Configurazione Form & Griglie	UICONF	Da business
Personalizzazione del DB	PROVA.MODXP(script .sql)	SQL server o <u>strumento di amministrazione</u>

Tramite l'editing è possibile aggiungere dei controlli a livello di UI, sia per componenti standard BN che personalizzati BO, ...

Va fatto in fase di esecuzione di business

Le configurazioni effettuate vengono salvate su UICONF

Si attiva dalla voce del menù attivabile dalla barra del titolo **Abilita\disabilita editing**

La personalizzazione delle Form\Griglie si attiva con il Ctrl+Shift+Click SX su di un qualsiasi controllo editabile del programma in esecuzione

Questa fase di personalizzazione permette di configurare i controlli di UI (anche quelli aggiunti tramite editing) su alcune proprietà:

visibile,
enable,
default, ...

GESTIONE SICUREZZA / CONFIGURAZIONE ACCESSI <PROVA - AZIENDA DI PROV...

Trova (Alt+F1)

CONTROLLI

Ricerca

Controllo	Descrizione	Tipo	Personalizzato	Configurato
edCodart	Codice articolo	NTSInformatica.NTSText...	☐	☐
lbCodart	Codice articolo	NTSInformatica.NTSLabel	☐	☐
edFocus		NTSInformatica.NTSText...	☐	☐
tlbPanelLeft		NTSInformatica.NTSBarB...	☐	☐

(1) Griglia Generale (3) Dipende Generale Ruolo Amministratori

(2) Elemento Generale (4) Proprietà --- Operatore Admin

(5) Valore Azienda PROVA

Ditta PROVA Aggiungi

(6) VALORI IMPOSTATI

Dipendenza	Nome proprietà	Valore	Azienda	Ditta

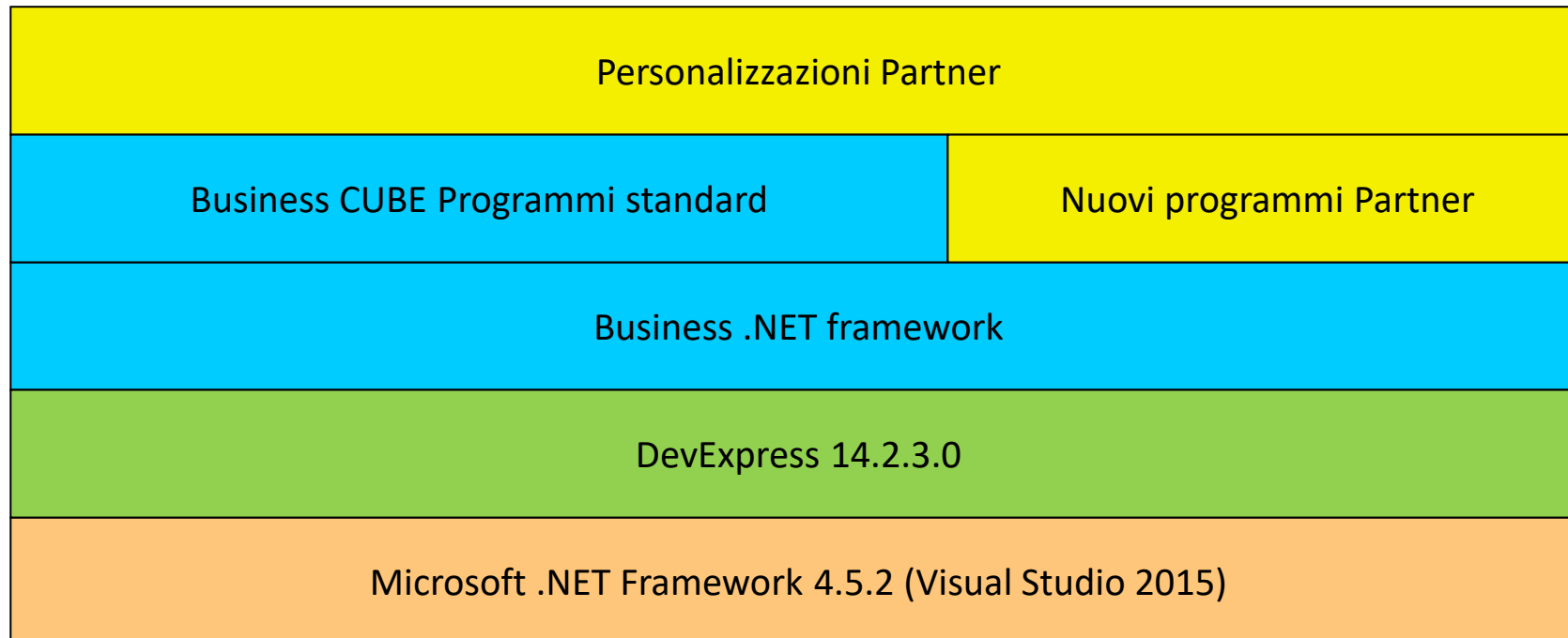
* Si ricorda che per LABEL collegate a tabelle non è possibile settare la proprietà "Testo"

Le proprietà VISIBILE/ABILITATO non operano se "da codice" gli oggetti sono stati dichiarati "non visibili" o "non editabili"

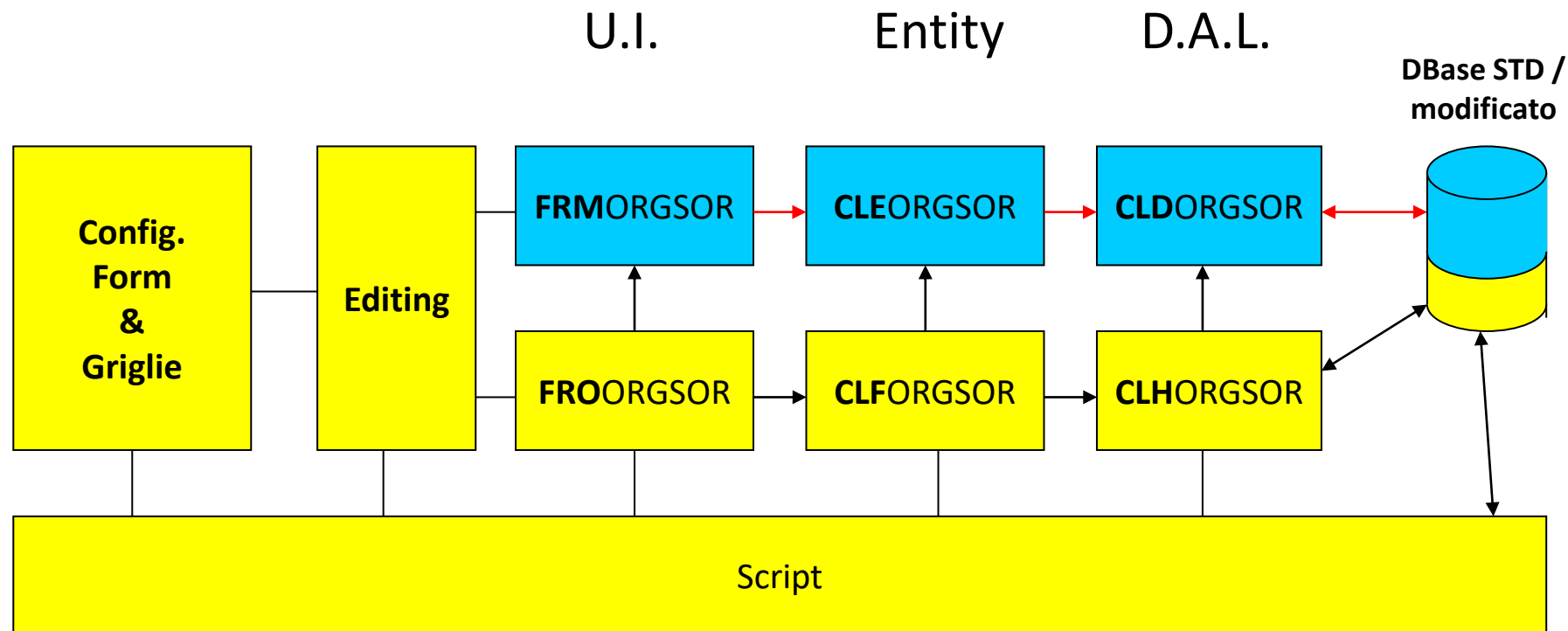
DIVERSO DA: per campi di tipo STRINGA o DATA se il valore deve essere diverso da "" indicare nella colonna valore "null"

TESTO settata su operatori con lingua diversa da Italiano: il contenuto del file .INT ha priorità rispetto al valore settato qui

Nascondi massivo



Relazione fra i vari strumenti di personalizzazione



Per configurare uno script, basta creare un file di testo con estensione .NTS.

Quando si lancia un child, be__menu lo istanzia, ne fa l'init, poi verifica se nella dir Script del server (una dir allo stesso livello della Rpt dir) è presente un file con lo stesso nome del child da lanciare, ma con estensione .NTS.

Se lo trova cerca di compilarlo e se tutto ok passa questo script al child di tipo 'UI' senza bisogno che, da codice, venga scritta alcuna riga particolare. Se il file non verrà compilato verrà scatenato un errore, ma il child standard verrà lanciato/eseguito normalmente (ovviamente le funzioni contenute nello script non verranno eseguite ...).

Una volta istanziato lo script, l'UI lo passa all'entity (questo in fase di creazione/attivazione entità) e al dal.

Una volta modificato\creato il file di script, per renderlo attivo basta uscire e rientrare dal programma (UI) che lo chiama, non è necessario uscire e rientrare da Bus.net.

Realizzazione degli esempi presenti nel file CT-0907-0027-Script e source extender

1. Script in Bus.net

1.1. Esempio di file di script completo

2. Esempi Script complessi in Bus.net

2.1. Esempio campo aggiuntivo (in anagrafica clienti) e relativa decodifica

2.2. Esempio gestione controlli inseriti in contenitori

2.3. Esempio gestione controlli su più form

2.4. Esempio di aggiunta di un nuovo evento

1.1. Esempio di file di script completo

- Serve a capire il funzionamento e la struttura di uno script.
- Import
- Exec
- Initcontrols (frmParent)
- Eventi
- Uso del BN__CLIE.NTS

2.1. Esempio campo aggiuntivo (in anagrafica clienti) e relativa decodifica

- Primo approccio all'editing con l'aggiunta di 3 controlli
- Creazione di uno script un po' più complesso che tratti più eventi
- Tratteremo la decodifica del campo divisione (accesso al database)

- Impostare da abilita/disabilita editing la label e il controllo textbox per la divisione
- Visualizzare da funzione visualizza datatable il nome TABELLA.CAMPO per inserire il textbox per la divisione
- Inserire una nuova label per la decodifica (nome controllo a piacere)
- Inserire il file BN__CLIE.NTS per identificare la validating del campo textbox della divisione aggiunto
- Identificata la funzione nello script NTS usare il template per codificare la nuova funzione
- Nella nuova funzione (per recuperare il controllo dedito alla decodifica) usare la:
- Funzione che ritorna il controllo di una form, ricercabile tramite nome
- `Public Function NTSFindControlByName(ByVal CtrlStart As Control, ByVal strControlName As String) As Object`
- Negli script per avere la form usare l'oggetto frmParent

2.2. Esempio gestione controlli inseriti in contenitori

- Come usare i vari controlli presenti in maschera:
- `frmParent.NtsBarManager1.Items("tlbProva")`
- `frmParent.NTSFindControlByName(frmParent, "lbProva")`
- `frmParent.grvCiva.columns("tb_codciva")`

2.3. Esempio gestione controlli su più form

- Lancio di uno script su classi con più form (padre + modali)
- Come identificarle nella initcontrols
- Come risolvere l'eventuale problematica di eventi uguali

2.4. Esempio di aggiunta di un nuovo evento

- Come aggiungere un evento che non è disponibile fra quelli presenti a standard

- Controlli (spunta usa script)
- Scripting sulle stampe
- Procedure Alert
- Procedure Import \ Export
- Procedure Configuratore

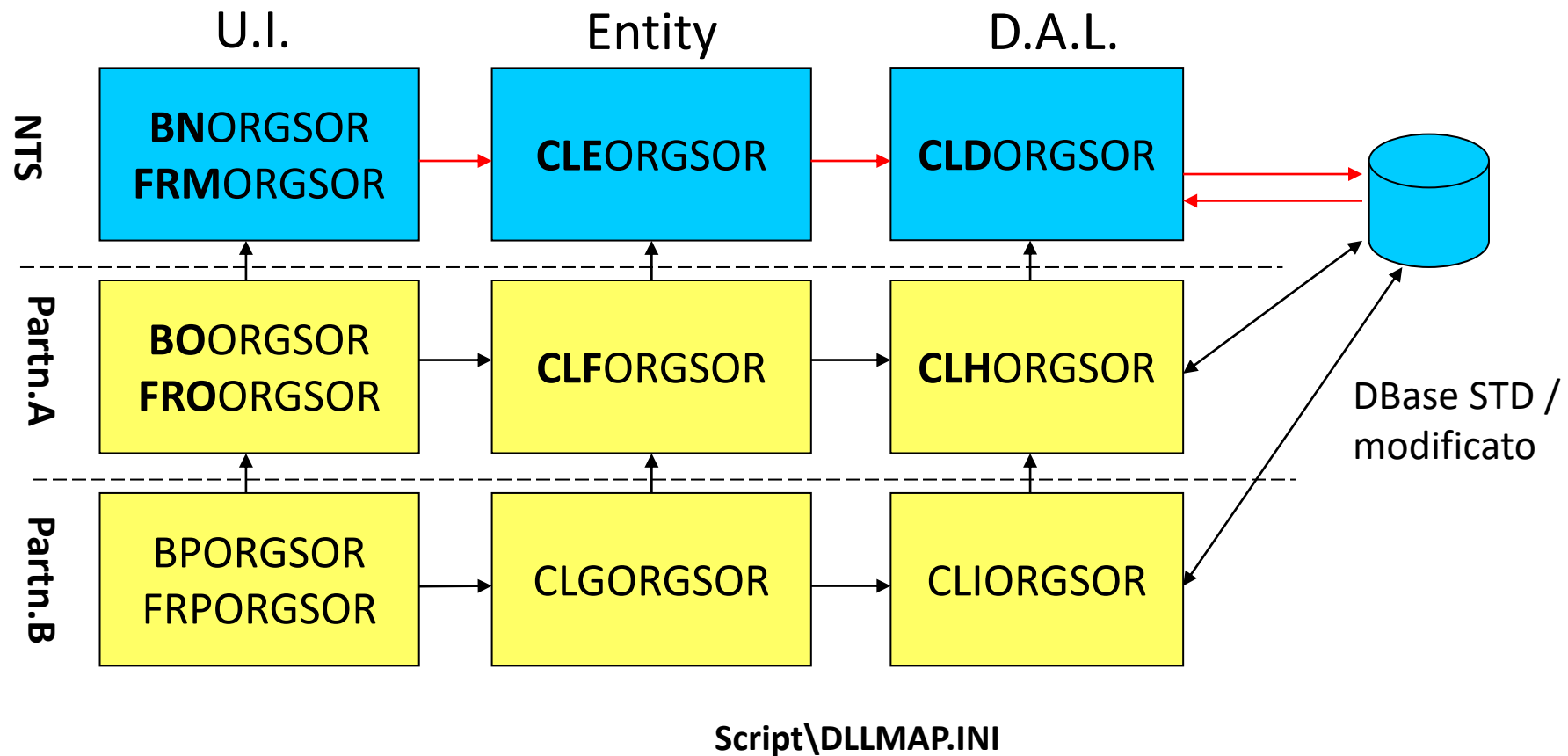
- Editing avanzato:

Controlli non trattati prima

- Configurazione di un controllo: dock, autosize, ... (finestra richiamabile con Ctrl+Click SX)
- Cancellare\Modificare un controllo
- Posizione e ridimensionamento con mouse
- Configuratore User Interface

- Utilizzo del DALCONF.INI, BN__TABE e BN__HLTB per la gestione di una tabella che rispetti lo standard TABHHxx, con i campi tb_codhhxx e tb_deshhxx

Verticali e personalizzazioni con l'ereditarietà dei tipi



*|BN__CLIE.FRM__CLIE|BO__CLIE|NTSInfromatica.FRO__CLIE

*|BE__CLIE|BO__CLIE|NTSInfromatica.CLF__CLIE

*|BD__CLIE|BO__CLIE|NTSInfromatica.CLH__CLIE

+BN__CLIE

+BE__CLIE

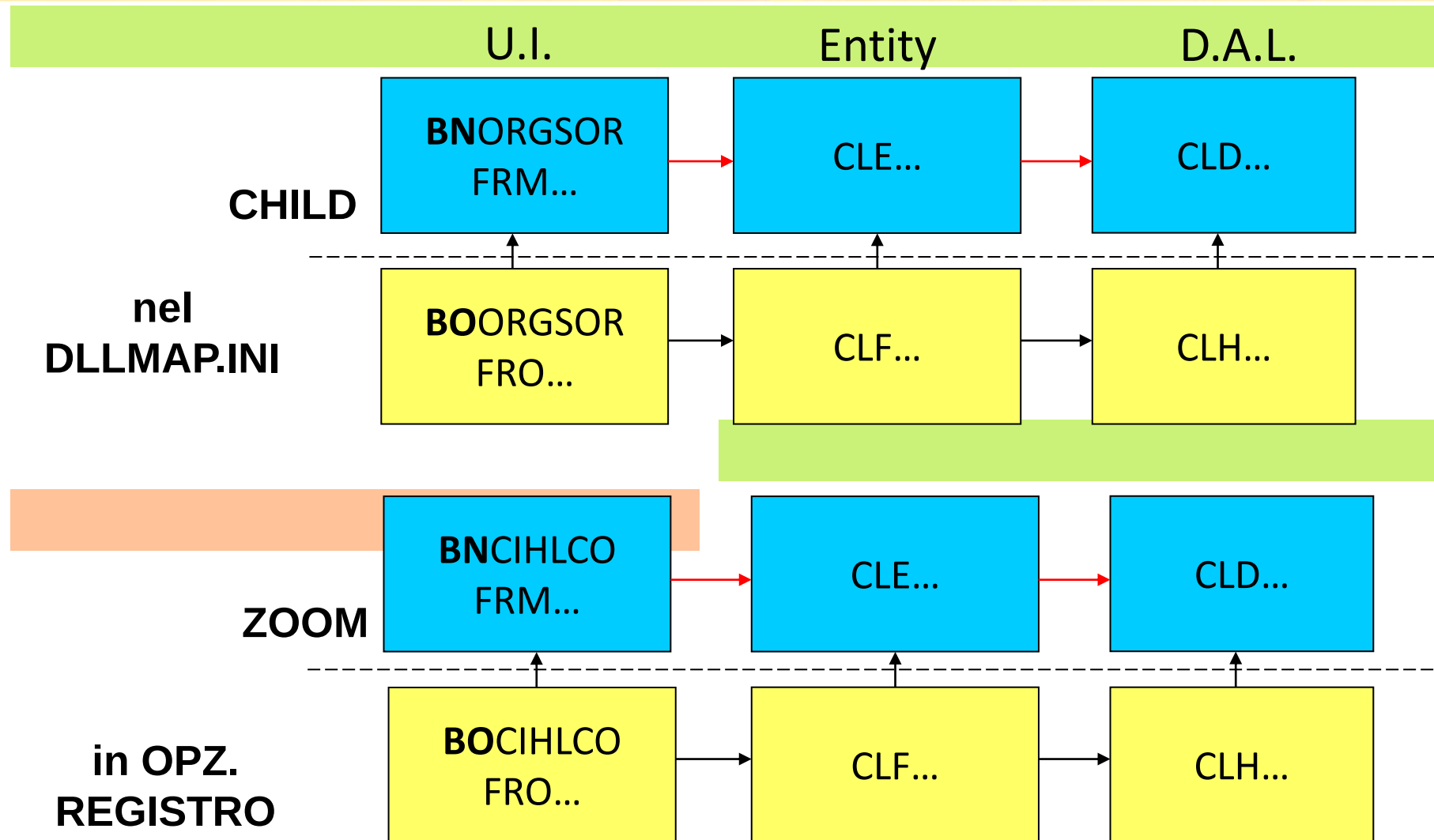
+BD__CLIE

+B*__CLIE

NB: solo per le U.I. degli zoom occorre indicare l'opzione ZOOM... Per esempio:

ZOOMTABHHVE = **BO** HHVE.**FRO** HHVE;BNHHCTVE

Come configurare i livelli personalizzati di child e zoom



Semplici:

Creazione dei componenti vuoti

Semplici implementazioni sui 3 livelli

Tecniche di overrides (mybase o copy-paste)

Referenziare controlli aggiunti ad editing e aggiunta di eventi

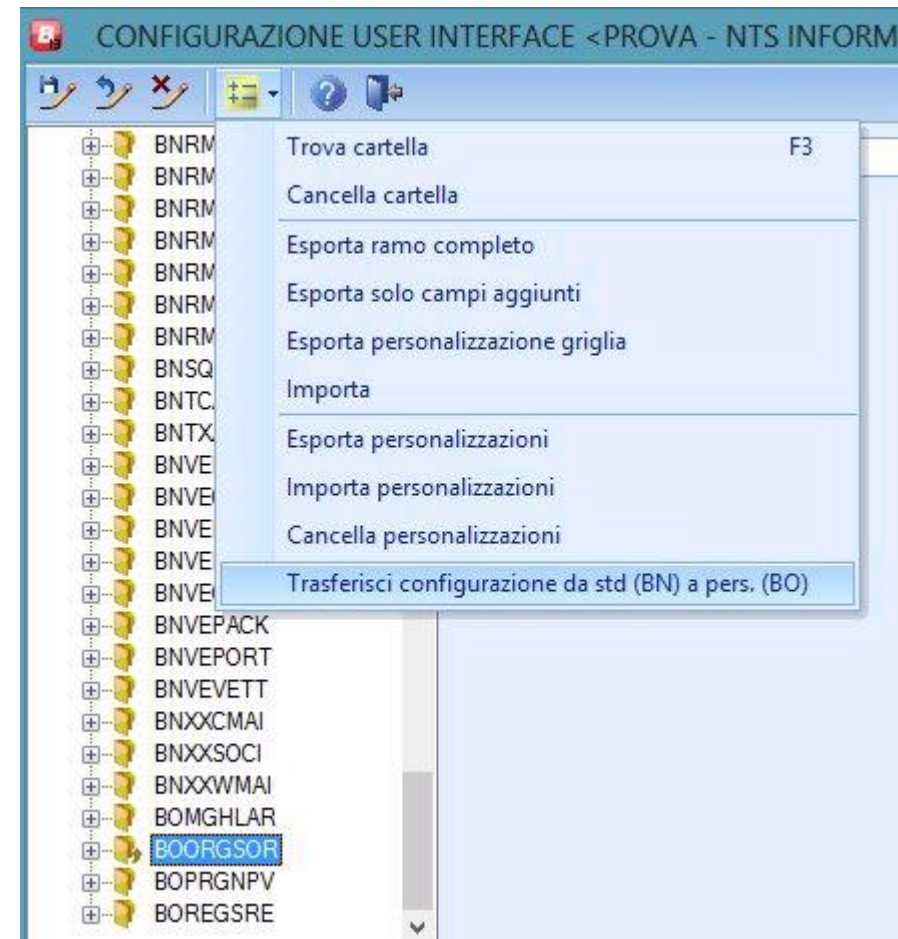
Complessi:

Aggiunta di una griglia in un BO

Uso degli entity Cleveboll/Cleorgsor per la creazione di documenti

Scrittura\lettura su un altro database nel DAL

Verrà fatto il trasferimento delle sole configurazioni che non sono già presenti nel BO.



C:\BIZ2017\BDCGPRIN\BDCGPRIN.VB (gestione prima nota):
Public Overridable Function Salva_Pers(

C:\BIZ2017\BDCGPRIN\BDCGPRIN.VB (gestione prima nota):
Public Overridable Function DeleteReg_Pers(

C:\BIZ2017\BDCRGSOFF\BDCRGSOFF.vb (gestione offerte):
Public Overridable Function SalvaOffertaBeforeChiudiTrans(

C:\BIZ2017\Cldorgsor\Cldorgsor.vb (gestione ordini):
Public Overridable Function SalvaOrdineBeforeChiudiTrans(

C:\BIZ2017\Cldveboll\Cldveboll.VB (gestione documenti):
Public Overridable Function SalvaDocumentoBeforeChiudiTrans(

- Per inserire in un modello di Word alcune informazioni personalizzate occorre ereditare sul BF__STWO la:
- Public Overrides Function SostituisciCampoPers(ByRef dtr As System.Data.DataRow, ByVal strCampo As String, ByRef strValore As String) As Boolean

- Funzioni chiamate prima dell'invio effettivo di e-mail e/o fax che permette di cambiare i dati contenuti nel documento da inviare (oggetto, destinatario, allegati, ...) o di eseguire invii ulteriori oltre a quello elaborato dalla routine standard
- **SEND_InvioFax**(ByRef strOggetto As String, ByRef strFileDaInviare As String, ByRef strFax As String, ByRef strDesconto As String, ByRef dttOrga As DataTable, ByVal strProgrChiamante As String)
- **SEND_InvioMail**(ByRef strTo As String, ByRef strToEx As String, ByRef strCC As String, ByRef strOggetto As String, ByRef strAllegati() As String, ByRef bConfermaLetturaDaClient As Boolean, ByRef strHtmlEmail As String, ByRef strTestoEmail As String, ByRef bModaleOutlook As Boolean, ByVal strProgrChiamante As String)
- Tutte le variabili (ad esclusione del programma chiamante) sono passate per riferimento e se il metodo ritorna "False" l'invio viene annullato.

Framework per TABLET

- Predisposto ambiente per poter creare form specifiche per tablet.
- Ora se Business parte con il parametro /MODE=IPAD o TOUCH o TOUCHBIGCHAR, ogni volta che viene lanciato un child viene cercata una form che si chiama come quella standard ma che termina per '_T': se presente viene avviata questa form, diversamente quella standard; in questo modo si può realizzare una form SPECIFICA per tablet. L'importante è che nella form specifica i parametri di quando viene chiamata da altro child siano gli stessi.
- NB: in questa modalità, OVVIAMENTE E VOLUTAMENTE, le personalizzazioni fatte con CTRL+SHIFT+CLICK, piuttosto che controlli estesi o script legati alla form standard non vengono utilizzati anche dalla form secondaria (che potrebbe avere un layout totalmente diverso, per cui le personalizzazioni potrebbero essere inapplicabili). In ogni modo anche in questa form si possono utilizzare gli strumenti canonici di personalizzazione delle maschere.
- Per personalizzare le maschere tramite editing (disattivato in modalità TOUCH) occorre entrare in Business normalmente, ma impostando per un l'utente con il quale si accede l'opzione generale OPZIONI\ForzaFormTablet = -1

Per fare in modo che eventuali personalizzazioni realizzate da partner che ereditano le funzioni standard possano continuare ad essere chiamate anche se nello standard sono state realizzate le stesse funzioni con passaggio di più parametri (rispetto a quelle ereditate dai partner).

Tramite questa funzione, nello standard, NTS aggiungerà parametri ad una funzione sempre accodandoli e nella nuova funzione le prime righe di codice dovranno essere:

```
Public Overridable Function ApriDocumento(ByVal strDitta As String, ByVal bNew As Boolean, ByVal strTipoDoc As String, _
                                         ByVal nAnno As Integer, ByVal strSerie As String, ByVal lNumdoc As Integer, _
                                         ByVal ds As DataSet, ByVal bModTCO As Boolean, ByVal bModPM As Boolean, ByVal nOrdri
nOrdri As Integer) As Boolean
    Dim strSQL As String = ""
    Dim dttImp As New DataTable
    Dim i As Integer = 0
    Dim dtrT() As DataRow = Nothing
    Dim strTestmag As String = "testmag"
    Dim strMovmag As String = "movmag"
    Try
        '-----
        'per compatibilità con funzioni ereditate da rive in versioni precedenti
        Dim oOut As Object = Nothing
        Dim oIn As New ArrayList(New Object() {strDitta, bNew, strTipoDoc, nAnno, strSerie, lNumdoc, _
                                                ds, bModTCO, bModPM, nOrdri})
        If CLN__STD.CheckInvokeCustomFunction(CustomClass, Me, System.Reflection.MethodInfo.GetCurrentMethod, oIn, oOut)
Then
            ds = CType(oIn(6), DataSet)
            Return CBool(oOut)
        End If
        '-----
        ...
    End Try
End Function
```

Nella routine CheckInvokeCustomFunction viene verificato se nella dll ereditata è presente la stessa funzione di quella chiamata ma con meno parametri, nel caso viene eseguita la funzione personalizzata e non quella standard.

Per chi sviluppa secondo gli standard Nts occorre porre particolare attenzione a:

- nella dichiarazione di oIn devono esser elencati TUTTI i parametri della funzione nell'ordine con cui sono impostati nella funzione
- nel caso che la CheckInvokeCustomFunction restituisca TRUE, oOut deve essere tipizzato in base al tipo di dato restituito dalla funzione

Con BNPDGENP è possibile inviare degli allegati oltre ai pdf generati, nel child che chiama FRMPDGENP va aggiunto del codice tipo quello sottostante (per un test era stato inserito in BNORCONF, oppure vedi BNDIGEDI), però settando gli allegati è possibile inviare solo via e-mail:

'test allegati (i nomi delle colonne del datatable sono fissi)

Dim dttAllegati As New DataTable

dttAllegati.Columns.Add("FILE", GetType(String))

dttAllegati.Columns.Add("KEY", GetType(String))

dttAllegati.Columns.Add("VALUE", GetType(String))

'i parametri KEY sono quelli contenuti nel datatable dei report da inviare,

'per un dettaglio valori vedi il contenuto della funzione FRMPDGENP.AggiungiColonneMancantiDatatable

'se si indicano delle strighe nei valori, devono essere passate con CSTRSQL(), per numeri con decimali CDBLSQL(), per date CDATASQL()

dttAllegati.Rows.Add(New Object() {"c:\tmp\a.txt", "Conto\$Agente", "4010107\$0"})

dttAllegati.Rows.Add(New Object() {"c:\tmp\b.txt", "Conto", "4010001"})

dttAllegati.Rows.Add(New Object() {"c:\tmp\a.txt", "Conto\$Agente", "4010001\$4"})

dttAllegati.AcceptChanges()

oPar = New CLE__CLDP

oPar.Ditta = DittaCorrente

oPar.strPar1 = "BSORCONF"

oPar.strPar2 = strQueryCrw32FileMultipli

oPar.strPar3 = strQueryCrw32FileUnico

oPar.strPar4 = strQueryGetDocMultipli

oPar.strPar5 = strQueryGetDocUnico

oPar.strParam = "Stampa ordini"

oPar.ctlPar1 = Me

oPar.ctlPar2 = dttFormule

oPar.ctlPar3 = dttAllegati

Grazie per l'attenzione

Filippucci Fabrizio